

고급머신러닝 보고서

합성곱신경망

Convolutional Neural Network

전자공학부 전자공학과
2020144005 김윤성
2024. 10. 20 제출

목차 INDEX

01 사전 지식

- 합성곱
- 이미지
- ANN의 문제점
- 신경망 + 합성곱

02 합성곱신경망

- CNN의 개념
- CNN
- CNN Architecture
- Datasets

03 실습 #1)

- CNN by keras
- 코드 분석
- 실행 결과

04 실습 #2)

- 규칙 & 정보
- 코드 분석
- 실행 결과
- 분석 내용

05 실습 #3)

- Cifar - 10
- 코드 분석
- 실행 결과
- 분석 내용

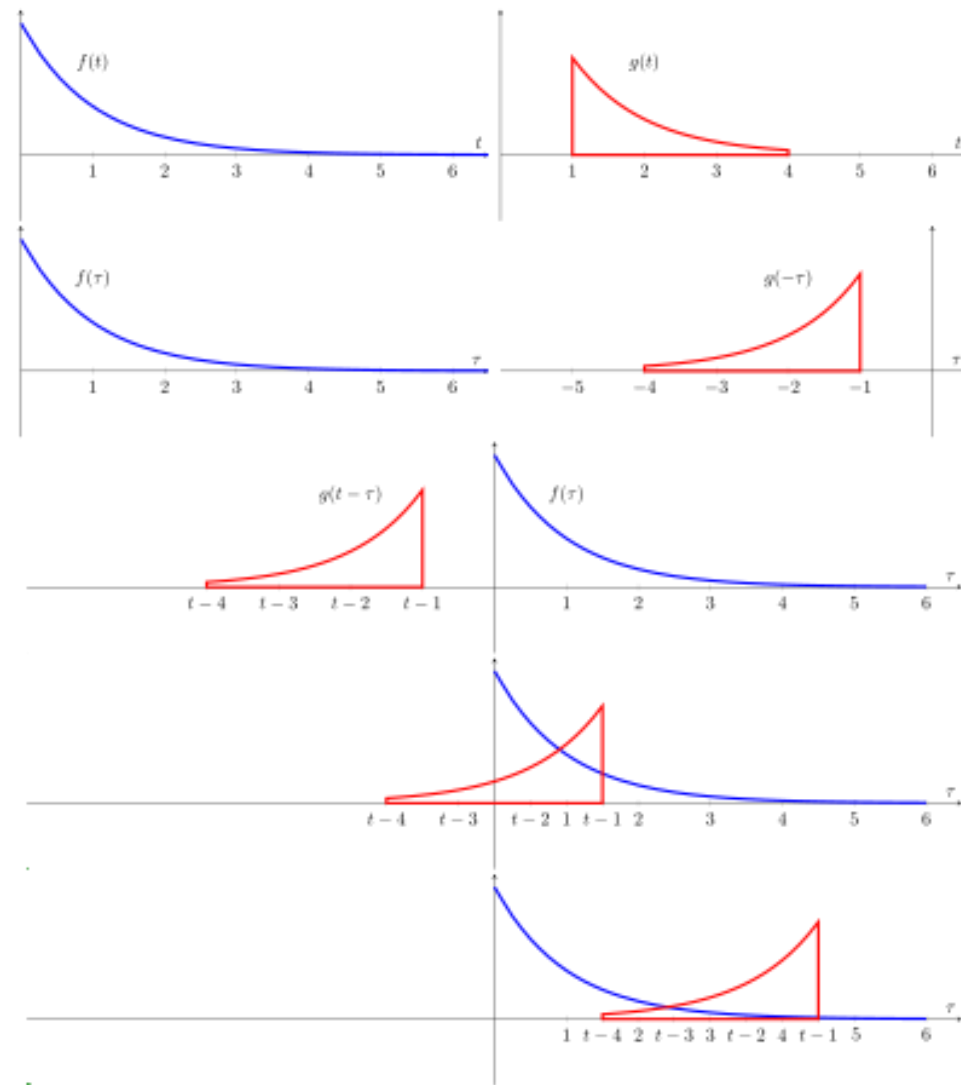
01

사전 지식

Background Knowledge

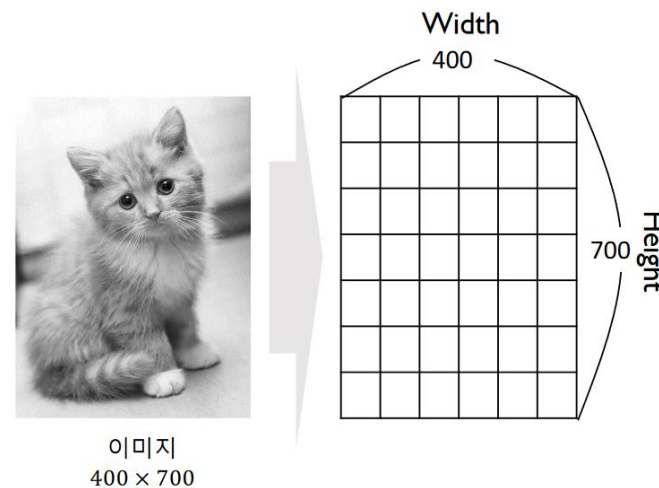
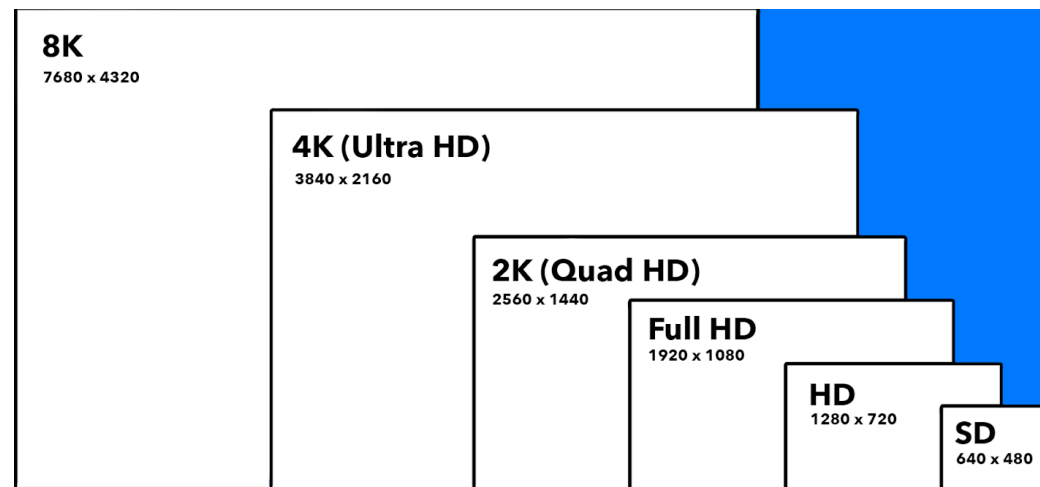
합성곱 Convolution

- 하나의 함수와 또 다른 함수를 반전한 후 이동한 값을 곱한 다음, 구간에 대해 적분하여 새로운 함수를 구하는 수학 연산자
- 정의: $f * g = \int_{-\infty}^{\infty} f(\tau)g(t - \tau)d\tau$



이미지 Image

- 2차원 평면에서 색상과 명암을 통해 시각적인 정보를 표현한 데이터
- 고유한 색상값과 명암을 가진 픽셀들로 구성
- RGB 모델을 사용하는 컬러 이미지의 경우, 원본 색상을 빨강, 초록, 파랑의 3가지 속성으로 분할 후, 각각의 색이 얼마나 포함되었는지를 나타내며, $0 \sim 2^8 - 1$ 의 범위를 가짐
- RGB, CMYK, HSV 등의 다양한 색상 모델 존재



인공신경망의 문제점 ANN's Problem

- 입력으로 1차원의 데이터만 받을 수 있음
- 2차원 이상의 데이터를 자르고 붙여서 1차원 벡터로 Flatten
- Flatten 시 공간에 대한 정보가 유실, 비효율적 학습



Flatten 전 2차원 이미지



Flatten 후 1차원 벡터
(실제로는 데이터를 가진 최소단위인 픽셀단위로 Flatten 된다.)

신경망 + 합성곱 ANN + Convolution

- **Filter(Kernel)**: 작은 크기의 행렬로, 입력 데이터에 슬라이딩하면서 특정 패턴이나 특징을 추출
- **Sliding**: 필터가 입력 데이터 위에서 겹치지 않게 이동하면서 해당 픽셀과 필터 간의 곱셈 후, 결과를 모두 더함
- **Feature Map**: 필터가 입력 데이터에서 찾아낸 특징(연산 결과)으로 이루어진 출력 데이터
- **Padding**: 이미지 경계 부분의 데이터도 중요하게 반영하기 위해 입력의 외곽에 추가되는 데이터들. 주로 0으로 Padding하며, 이를 Zero-Padding이라 한다.
- **Stride**: 필터가 이동하는 간격

1x1	1x0	1x1	0	0
0x0	1x1	1x0	1	0
0x1	0x0	1x1	1	1
0	0	1	1	0
0	1	1	0	0

4		

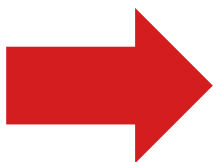
02

합성곱신경망

Convolutional Neural Network

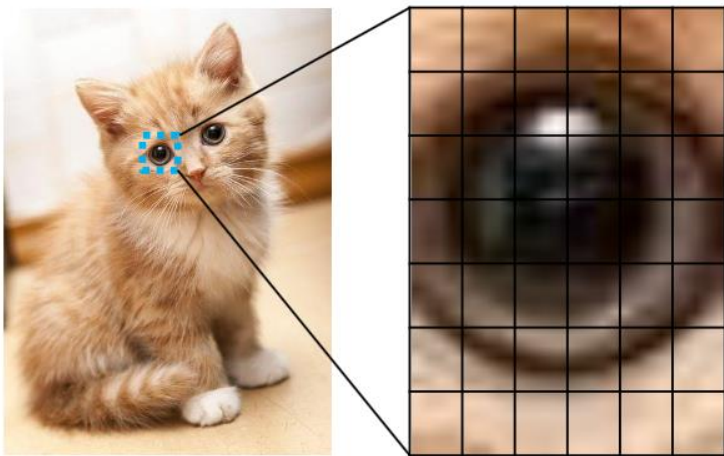
기본적 전략 Basic Strategy of CNN

- 이미지는 일반적으로 인접해있는 변수(여기서는 픽셀)간 높은 상관관계를 가짐 (Spatially Local Correlation)
- 이미지의 부분적 특성(눈, 귀,...)은 고정되어있지 않음 (Invariant Feature)

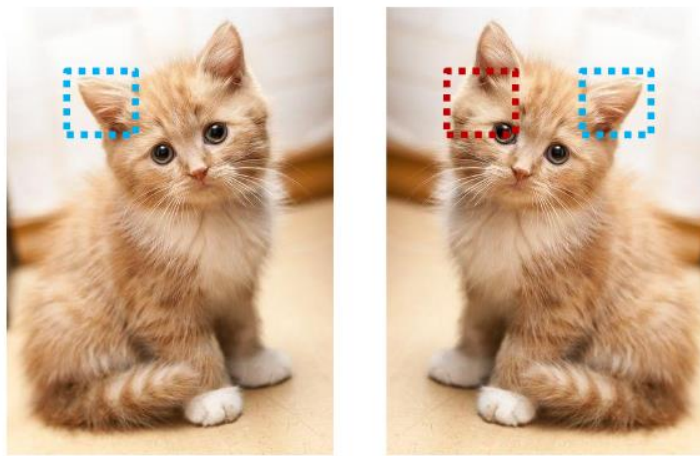


데이터의 특성을 반영한 모델을 구성하면
성능이 향상되지 않을까?

Spatially-Local Correlation

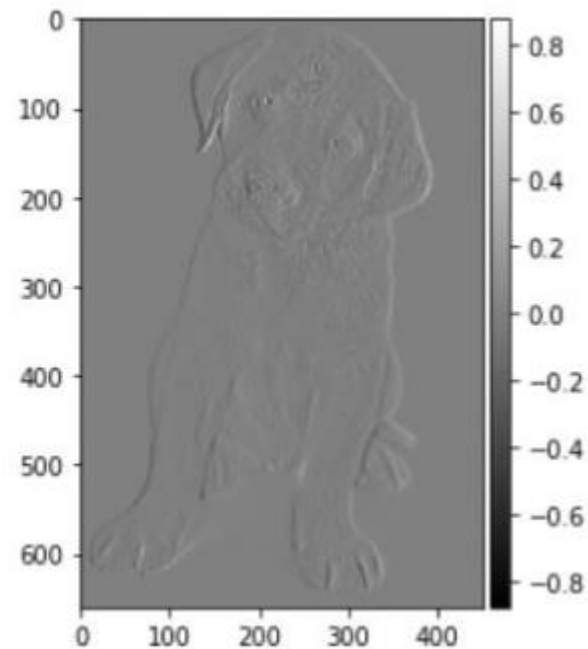
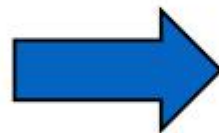


Invariant Feature



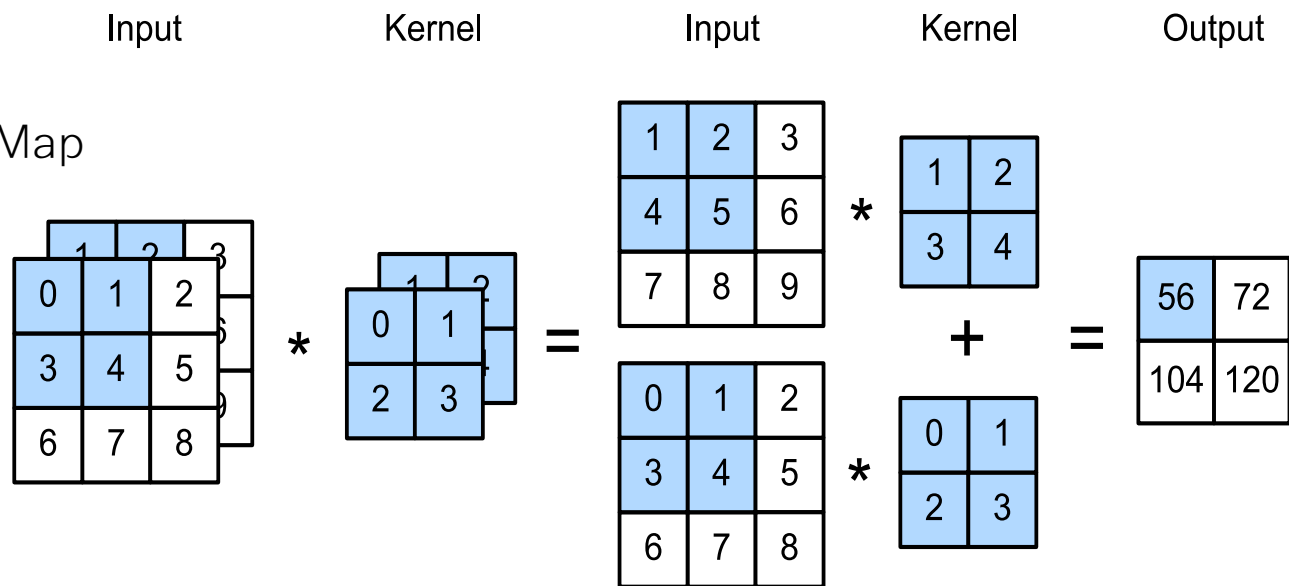
특징 추출 Extracting Edge Features

- 가장자리는 가장자리이기 때문에 본질적으로 색이 급격하게 바뀜
- 객체의 범위가 어디까지인지 쉽게 알 수 있음



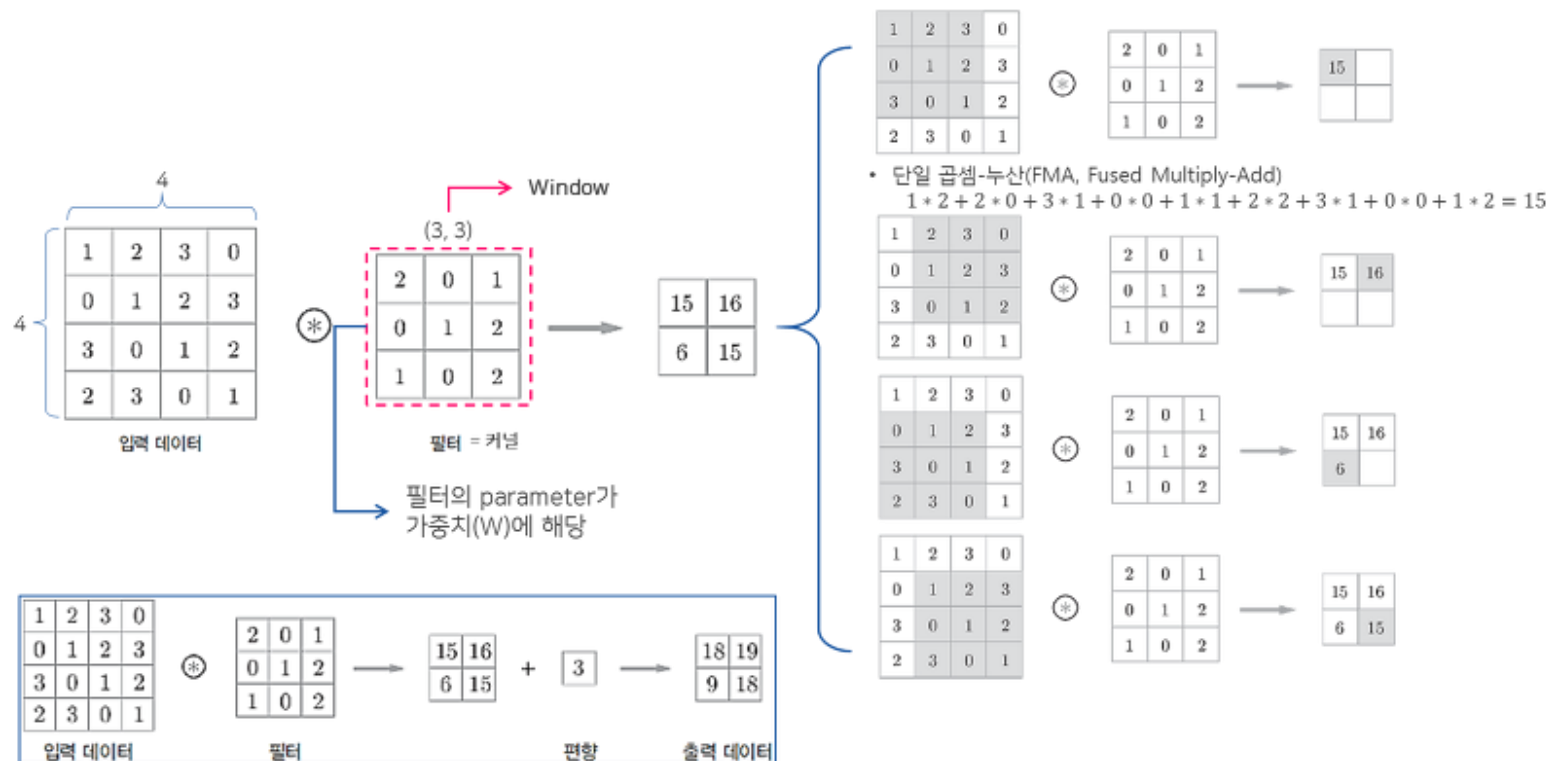
다채널 합성곱 Multi-Channel Convolution

- 만약 N개의 채널이라면, N개의 필터 사용
- 각 채널의 Feature Map을 모두 더해서 최종 Feature Map 사용 (최종 출력되는 Feature Map은 1개)
- N개의 필터를 사용하므로, 만약 입력이 2차원이라면 필터는 3차원, 필터의 집합은 4차원으로 볼 수 있음
- 각 채널에서 컨볼루션 연산 수행



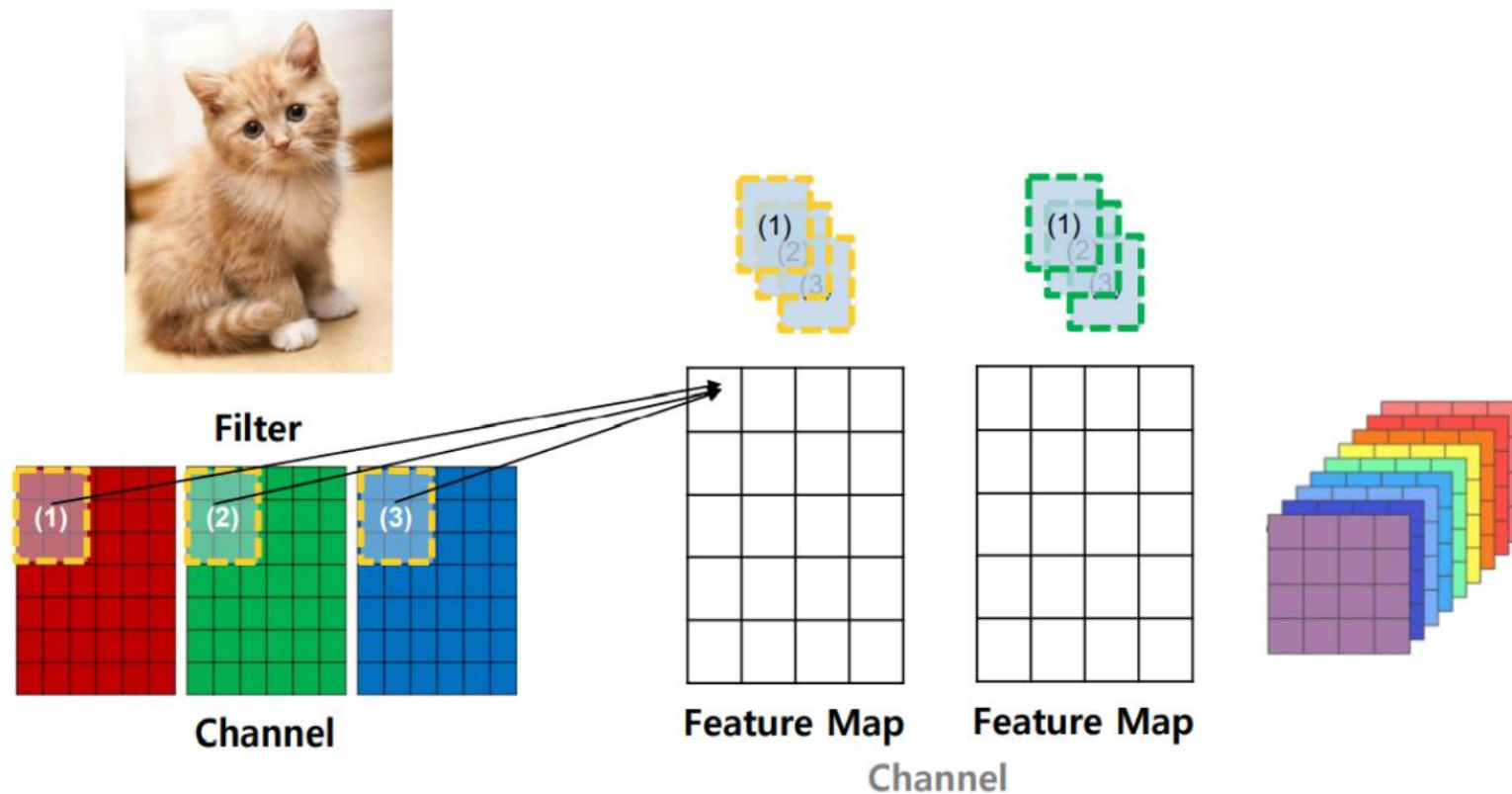
CNN Convolutional Neural Network

- 데이터와 필터의 모양을 (높이, 너비)로 나타내고, 윈도우(Window)라고 부름
- 여기서 입력 데이터는 (4, 4), 필터는 (3, 3)이고, 필터가 바로 Conv Layer의 가중치에 해당
- 합성곱 연산은 입력데이터와 필터간에 서로 대응하는 원소끼리 곱한 후 총합을 구하게 되며, 이것을 Fused Multiply-Add(FMA)라고 함
- 마지막으로 bias는 필터를 적용한 후에 더해줌



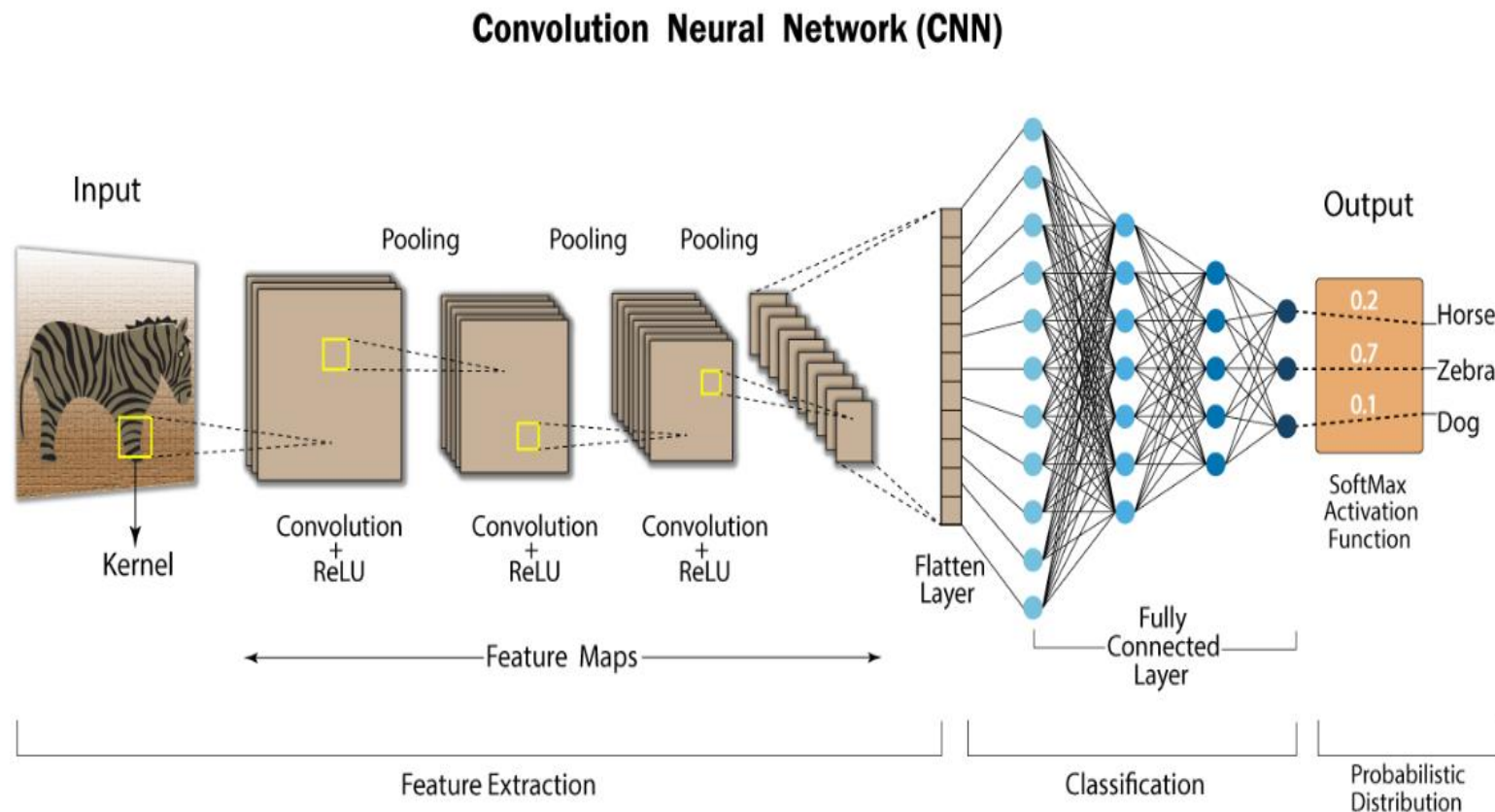
CNN Convolutional Neural Network

- 모든 채널로부터 얻은 결과를 합하여 Activation Function에 넣음
- Filter의 개수만큼 Feature Map 생성됨



CNN Convolutional Neural Network

- 입력의 본래 차원을 유지하면서 학습
- 일반적으로, CNN은 Convolution, Activation, Pooling의 반복
- 일정 횟수 이상 반복하면 Flatten되어 1차원의 벡터로 변환됨



패딩 Padding

- 가장자리의 픽셀들은 중앙에 위치한 픽셀들에 비해 필터와 만나는 횟수가 적음
= Convolution이 적게 수행됨
= 가장자리의 특징은 놓치기 쉬움
- 원본의 테두리에 임의의 픽셀 추가, 이것을 패딩이라고 부름
- 만약 0으로 채웠다면 Zero Padding이라 부름

3	1	1	2	8	4
1	0	7	3	2	6
2	3	5	1	1	3
1	4	1	2	6	5
3	2	1	3	7	2
9	2	6	2	5	1

0	0	0	0	0	0	0	0
0	3	1	1	2	8	4	0
0	1	0	7	3	2	6	0
0	2	3	5	1	1	3	0
0	1	4	1	2	6	5	0
0	3	2	1	3	7	2	0
0	9	2	6	2	5	1	0
0	0	0	0	0	0	0	0

Zero Padding
with size 1

풀링 Pooling

- 쉽게말해, Feature Map 압축하기
- 설정된 사이즈에 맞춰 주변 픽셀들을 뭉개버림
- Max Pooling: pool size중 가장 큰 값으로 이미지 처리에서는 Max Pooling을 주로 씀
- Average Pooling: pool size의 평균값으로
- Min Pooling: pool size의 최솟값으로
- 이 외에도 L2 Pooling, Global Pooling 등 다양한 방법이 있음

Max Pooling

29	15	28	184
0	100	70	38
12	12	7	2
12	12	45	6

2 x 2
pool size

100	184
12	45

Average Pooling

31	15	28	184
0	100	70	38
12	12	7	2
12	12	45	6

2 x 2
pool size

36	80
12	15

Flattening

- 다차원의 행렬 or 텐서 구조를 1차원의 벡터로 변환하는 과정

1	1	0
4	2	1
0	2	1

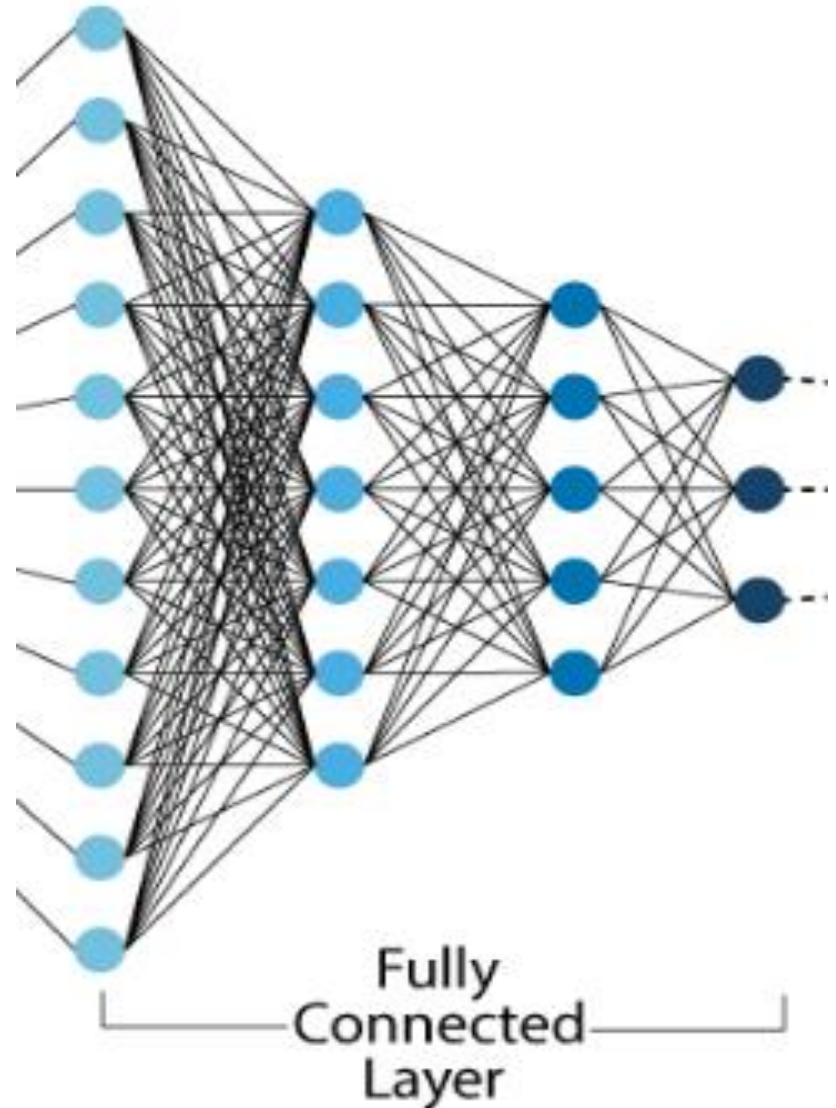
Pooled Feature Map

Flattening

1
1
0
4
2
1
0
2
1

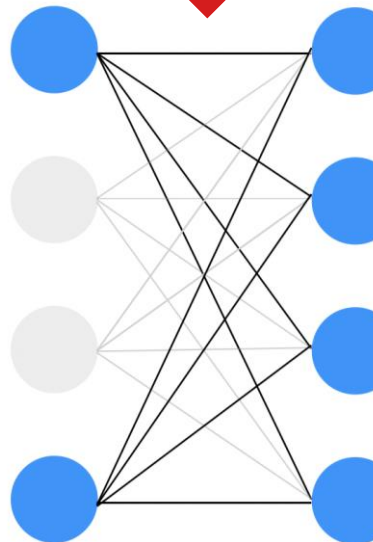
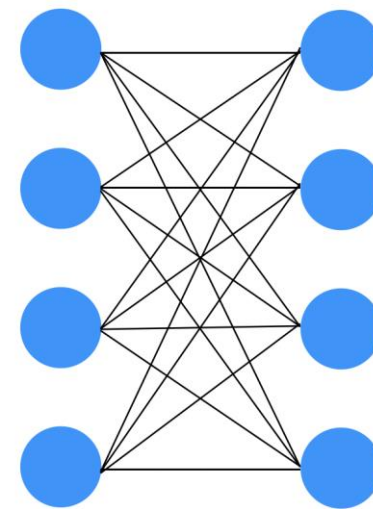
FC Layer Fully Connected Layer

- 쉽게말해, 모든 노드가 연결되어있는 신경망
- Convolution, Pooling 등을 거치며 추출된 특징을 기반으로 분류나 예측을 하기 위해 필요
- FC 레이어는 모든 노드가 연결되어있기 때문에, 작은 영역으로만 분석하는 Convolution과 달리 전역적으로 학습 가능



Dropout

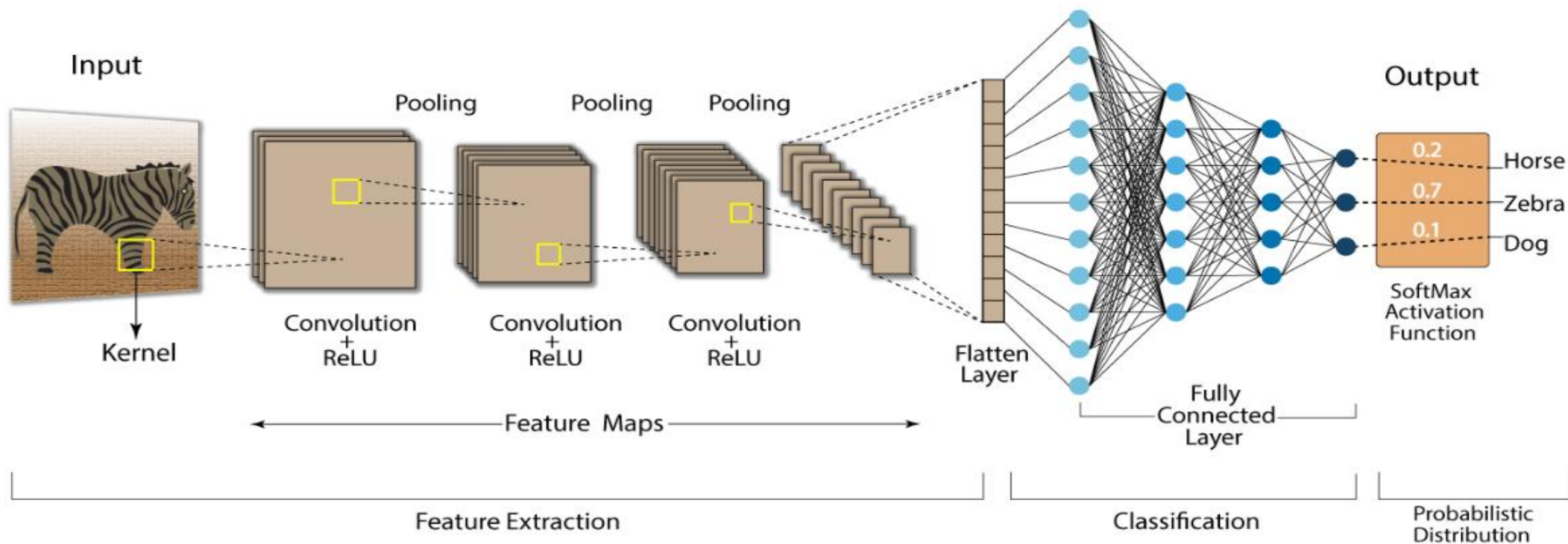
- 주로 딥러닝 모델에서 사용되는 기법으로, 학습 중 무작위 뉴런을 날려버림
- 이렇게 하면 놀랍게도 과적합이 방지되면서 일반화 성능이 증가함
- 모델이 특정 뉴런에 의존하지 않게 되기 때문
- dropout은 훈련 중에만 적용되고, 예측 시에는 모든 뉴런으로 예측함



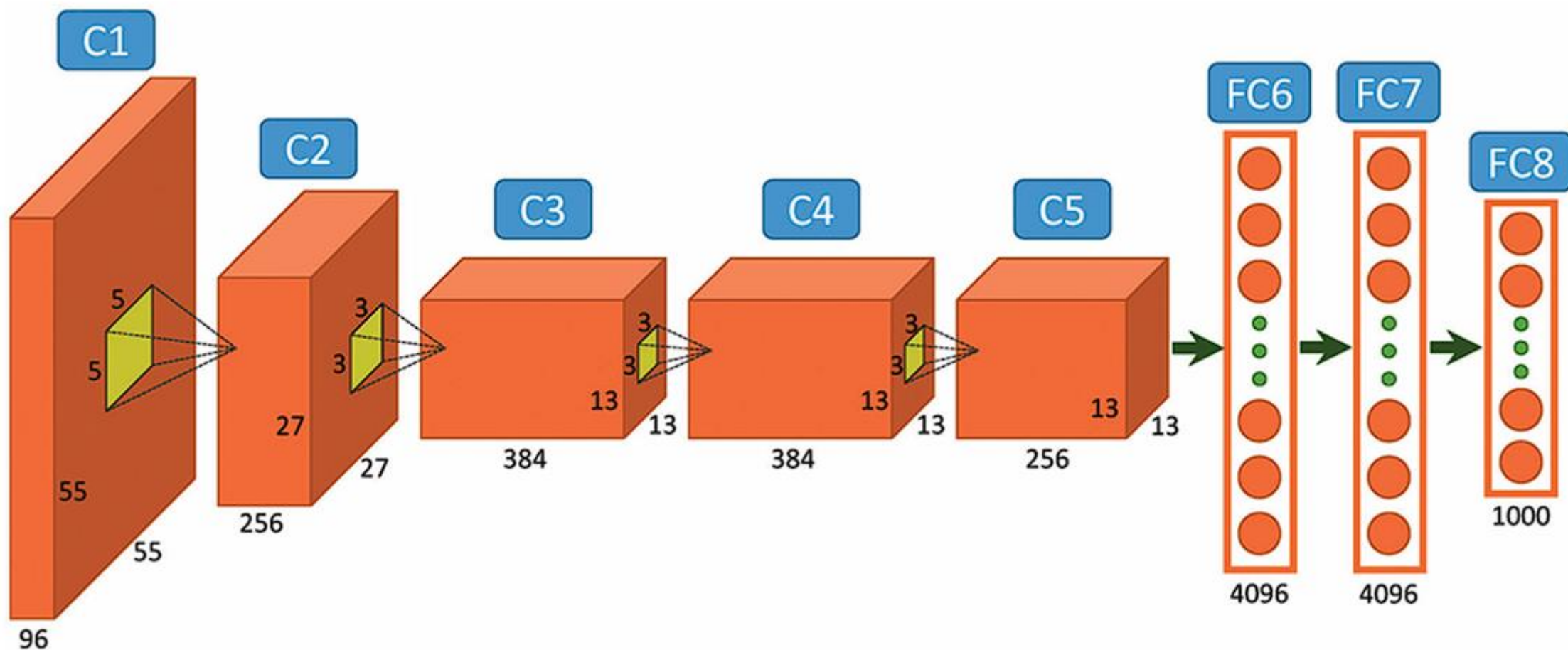
CNN Convolutional Neural Network

- Convolution 연산과 FC레이어의 구조를 잘 조정하면 뛰어난 성능을 얻을 수 있음

Convolution Neural Network (CNN)



AlexNet



AlexNet

- 8개의 레이어로 이루어져 있으며, 앞 5개는 Convolution, 뒤 3개는 FC Layer

- 입력: 227×227 의 RGB 이미지 ($227 \times 227 \times 3$)

- C1: 11×11 의 필터 96개, stride = 4, padding = 0, 이후 Max Pooling(3×3) $55 \times 55 \times 96$

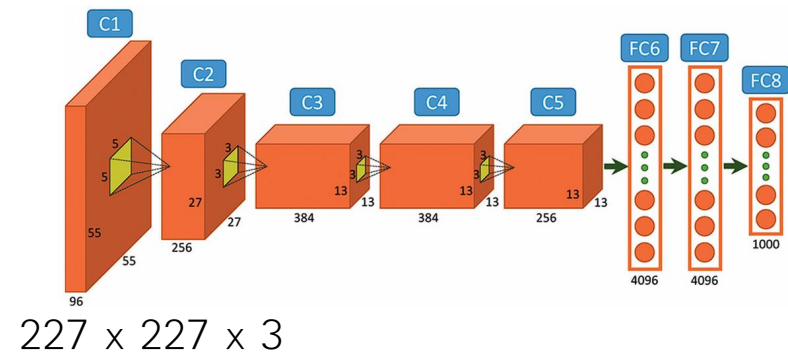
- C2: 5×5 의 필터 256개, stride = 1, padding = 2, 이후 Max Pooling(3×3) $27 \times 27 \times 96$

- C3: 3×3 의 필터 384개, stride = 1, padding = 1 $13 \times 13 \times 384$

- C4: 3×3 의 필터 384개, stride = 1, padding = 1 $13 \times 13 \times 384$

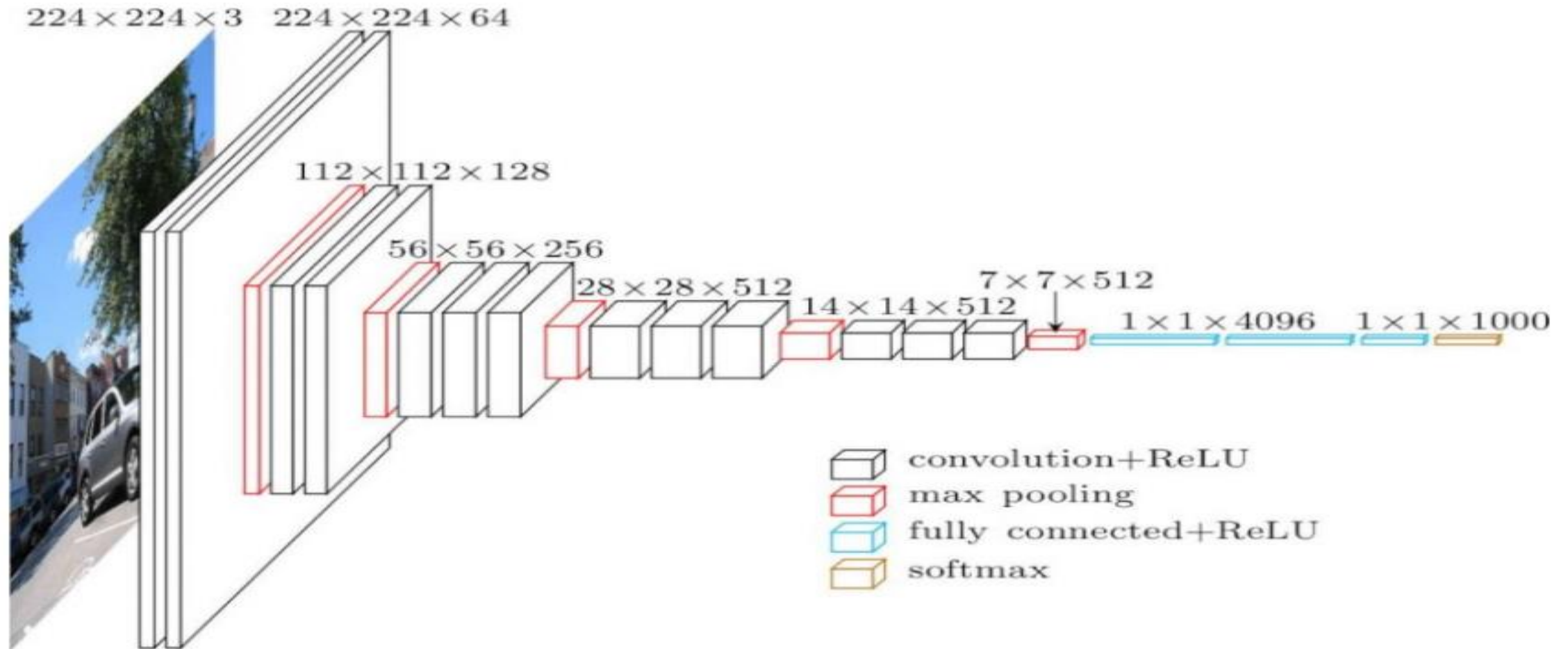
- C5: 3×3 의 필터 256개, stride = 1, padding = 1, 이후 Max Pooling(3×3) $13 \times 13 \times 256$

- FC6 ~ FC8: Flatten 후 1000개 클래스로, 출력 레이어에서는 softmax 사용



$6 \times 6 \times 256 \rightarrow 9216$ (Flatten) $\rightarrow$
 $4096 \rightarrow 4096 \rightarrow 1000$

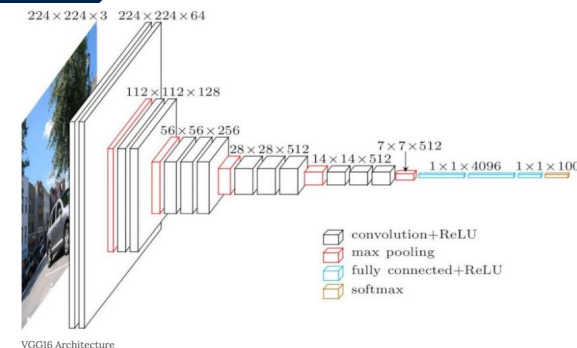
VGGNet Visual Geometry Group Network



VGG16 Architecture

VGGNet Visual Geometry Group Network

- 16개(VGG16) 또는 19개(VGG19)의 레이어를 가짐
- 입력: 224 x 224의 RGB 데이터
- L1, L2: 3 x 3의 필터 64개, stride = 1, padding = 1
- Pooling: Max Pooling(2 x 2), stride = 2
- L3, L4: 3 x 3의 필터 128개, stride = 1, padding = 1
- Pooling: Max Pooling(2 x 2), stride = 2
- L5, L6, L7: 3 x 3의 필터 256개, stride = 1, padding = 1
- Pooling: Max Pooling(2 x 2), stride = 2



224 x 224 x 3

224 x 224 x 64

112 x 112 x 128

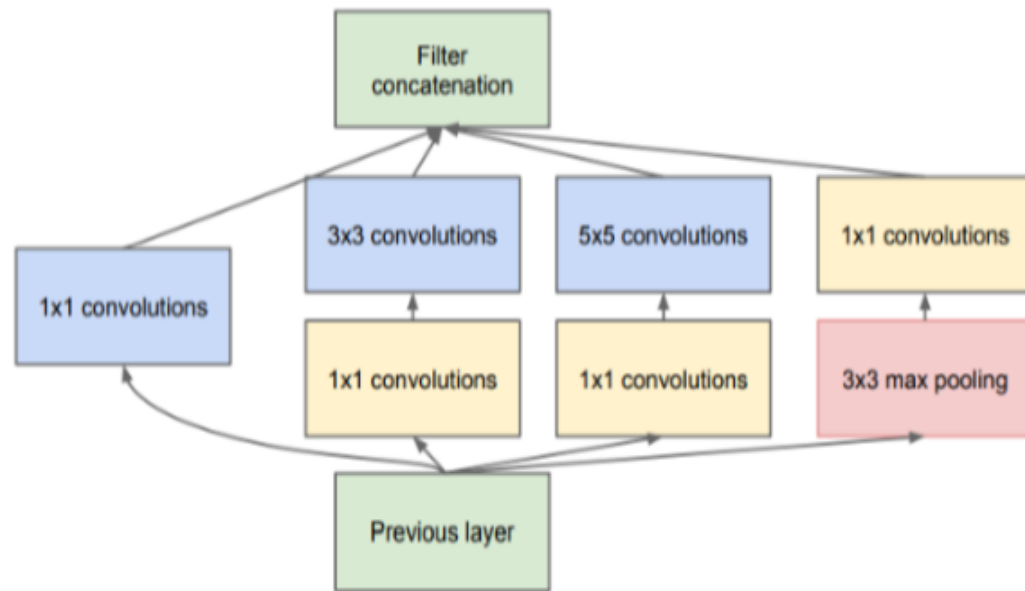
56 x 56 x 256

VGGNet Visual Geometry Group Network

- L8, L9, L10: 3 x 3의 필터 512개, stride = 1, padding = 1 28 x 28 x 512
- Pooling: Max Pooling(2 x 2), stride = 2
- L11, L12, L13: 3 x 3의 필터 512개, stride = 1, padding = 1 14 x 14 x 512
- Pooling: Max Pooling(2 x 2), stride = 2
- L14, L15, L16: FC, flatten 후 softmax를 이용해 1000클래스로 분류 7 x 7 x 512 -> 25088(Flatten) -> 4096 -> 4096 -> 1000
- VGG16은 Convolution 13개, FC Layer 3개로 구성됨
- VGG19는 Convolution 16개, FC Layer 3개로 구성됨

Inception

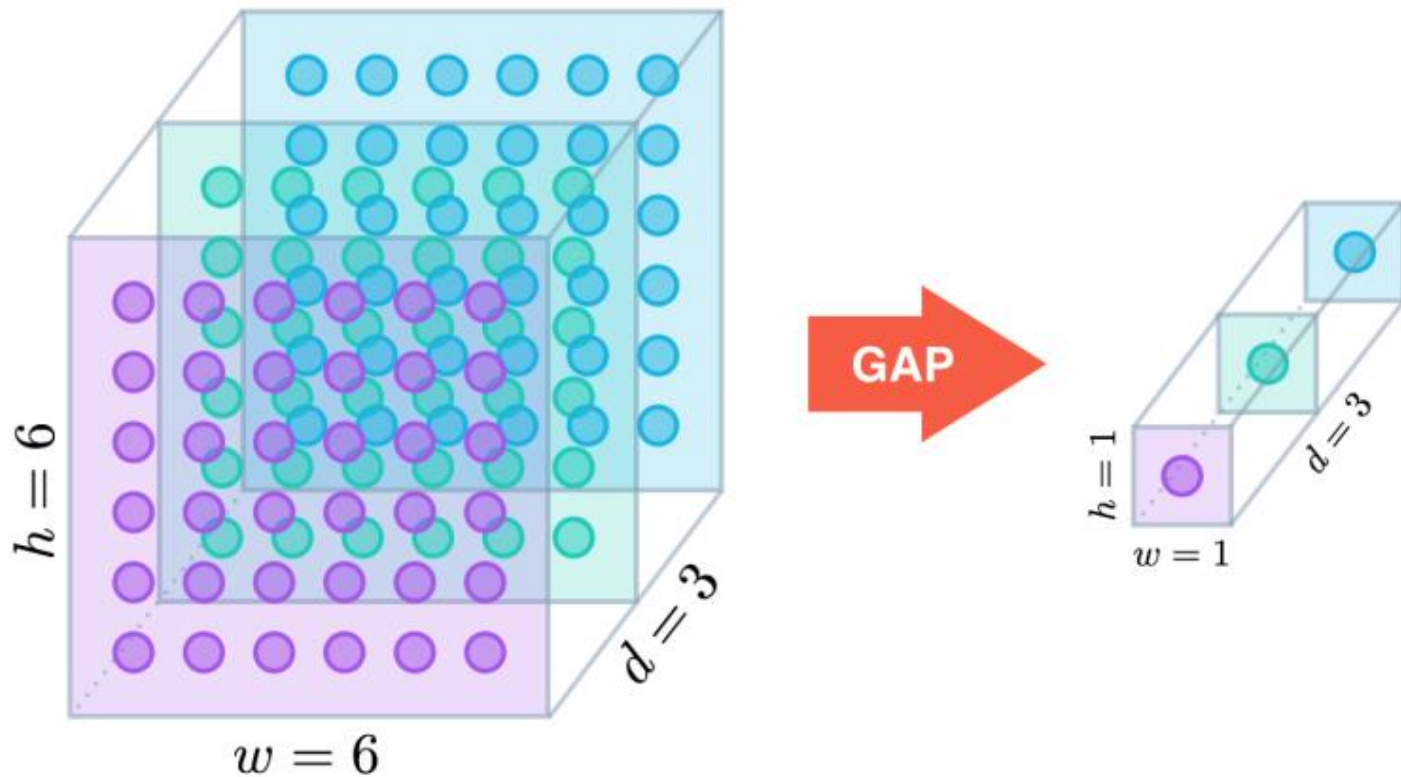
- GoogleNet 이라는 아키텍처를 구성하는 모듈
- 채널의 크기는 변경하지 않고, 개수만 조정 => 복잡도 감소 효과
- 입력을 여러 Convolution Layer에 넣은 후, 결과값들을 채널 방향으로 Concatenate 해서 출력함
- 다양한 크기의 필터를 이용하여 다양한 스케일에서 특징 추출
- 1 x 1 필터를 사용하여 차원 축소 => 연산량 감소 & 정보 유지
- 깊게 쌓아도 파라미터 수가 적음 => 성능 향상



GAP

Global Average Pooling

- 쉽게말해, Pooling을 하는데, 그 크기가 Feature Map 전체임
- 출력 차원의 엄청난 축소 가능
- 파라미터 감소, 과적합 방지 가능
- Feature Map의 전역적인 정보 반영 가능

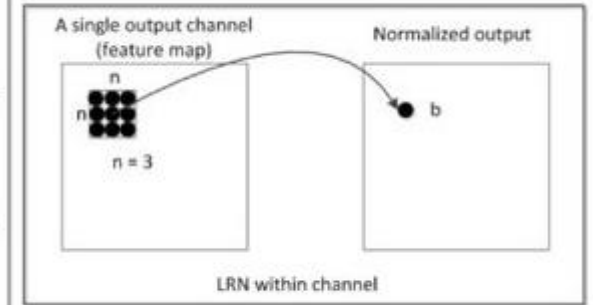
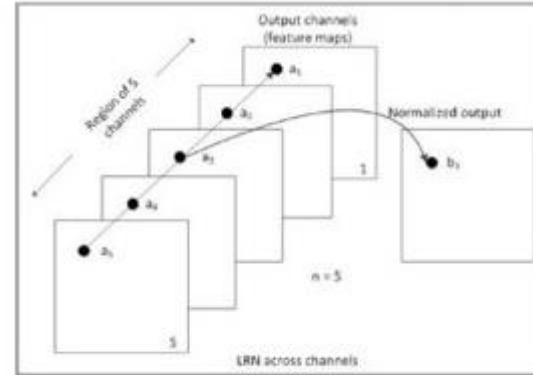


LRN Local Response Normalization

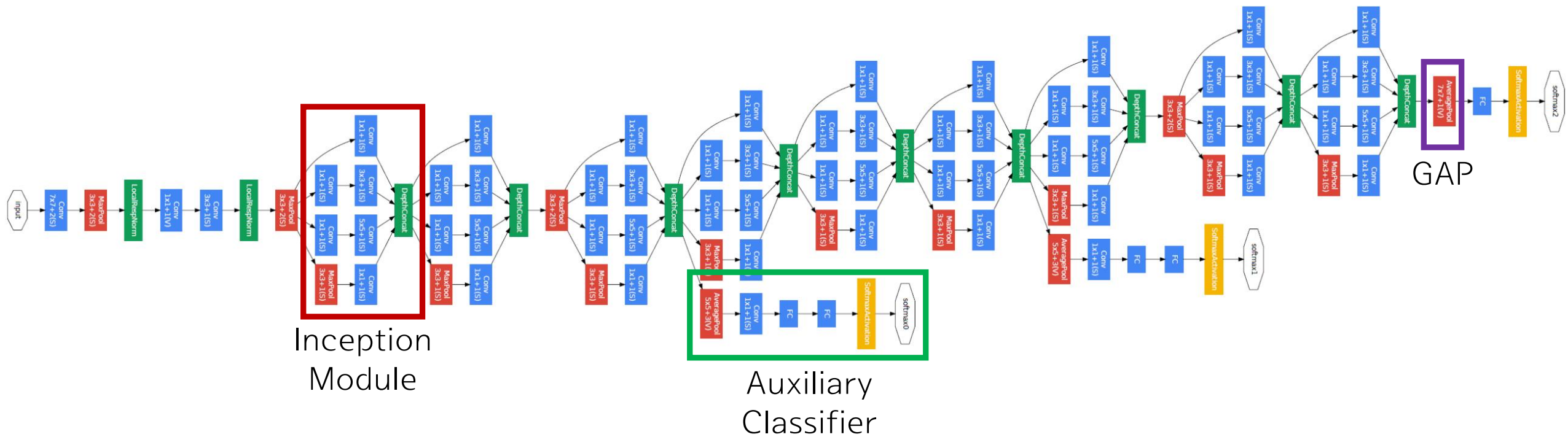
- 특정 뉴런의 활성값을 주변 뉴런과 비교하여 정규화
- ReLU는 입력이 양수이면 입력이 그대로 출력으로 나감 => Convolution 또는 Pooling시 높은 픽셀값이 주변에 영향끼침

$$b_{x,y}^i = \frac{a_{x,y}^i}{(k + \alpha \sum_{j=\max(0, i-\frac{n}{2})}^{\min(N-1, i+\frac{n}{2})} (a_{x,y}^j)^2)^\beta}$$

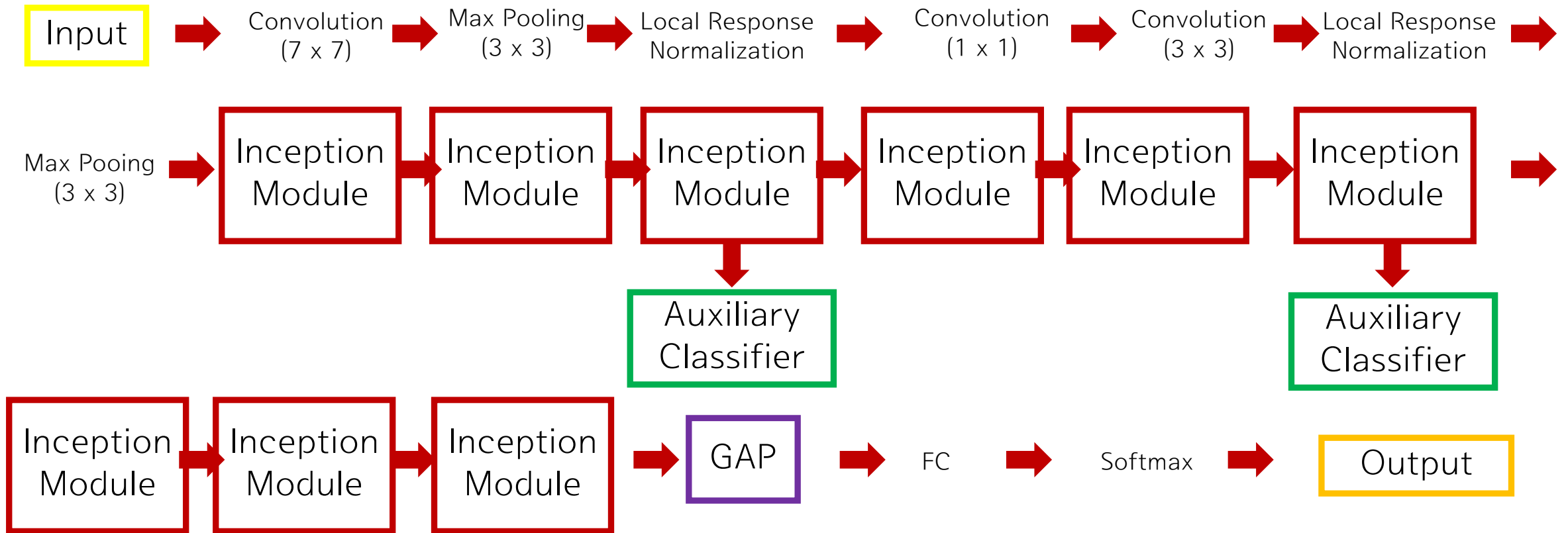
- $a_{x,y}^i$: 정규화되기 이전 뉴런의 값
- $b_{x,y}^i$: 정규화된 출력값
- k, α, β : 하이퍼 파라미터
- n : 정규화 과정에 참여하는 주변 뉴런의 수
- i : 채널 인덱스



GoogleNet

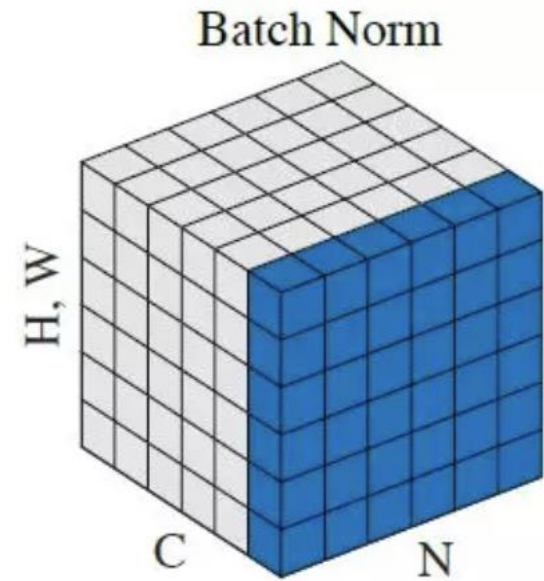


GoogleNet



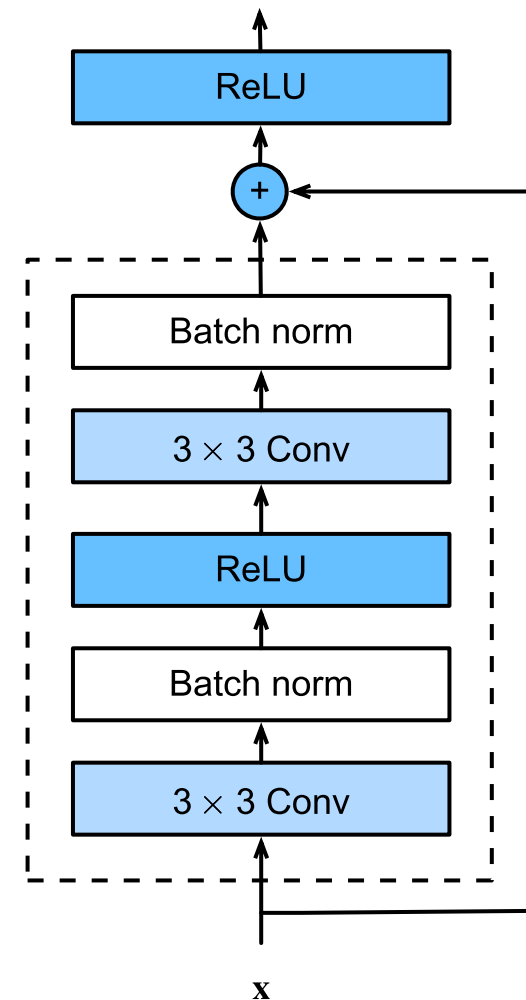
Batch Normalization

- LRN과 달리 각 레이어의 입력을 정규화
- Minibatch의 평균과 분산을 사용해 정규화
- 빠른 학습, 과적합 방지, 가중치 초기화에 덜 민감
- 계산량 증가, Minibatch가 작을수록 효과 감소
- $\chi = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$
- μ 는 Minibatch의 평균,
- σ^2 는 Minibatch의 분산,
- ϵ 은 분모가 0이 되는걸 방지하기 위한 매우 작은 값

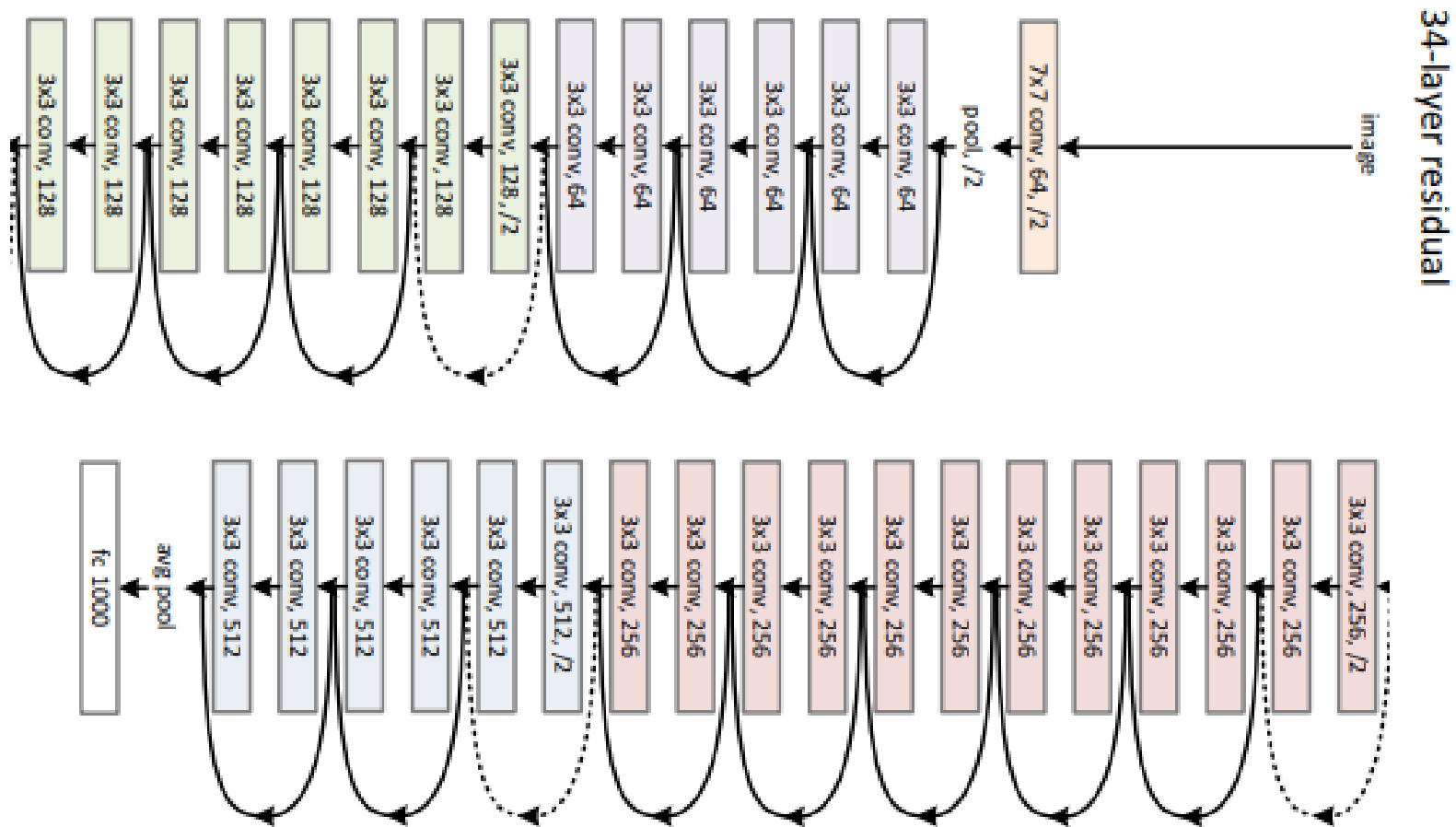


Residual Block

- ResNet 이라는 아키텍처를 구성하는 모듈
- 깊은 네트워크에서 Gradient Vanishing 문제 완화 가능
- 네트워크가 더 빠르게 수렴, 더 빠른 학습 가능
- 네트워크가 깊어져도 성능 향상 가능
- 입력과 Convolution을 거친 입력을 더해 출력됨

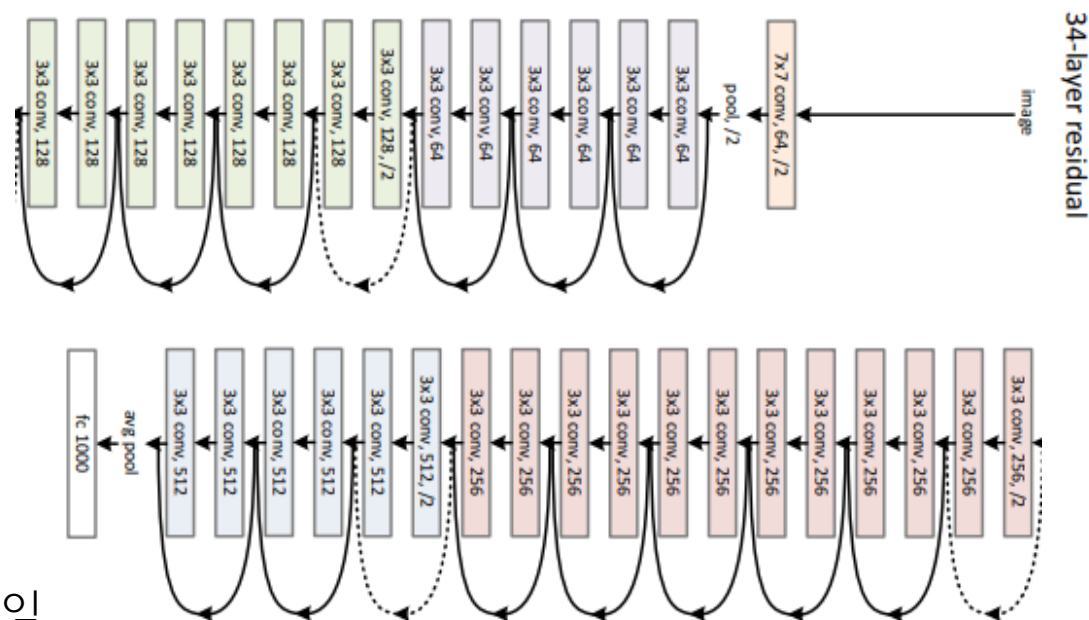


ResNet Residual Network



ResNet Residual Network

- 입력: 이미지와 같은 크기로 입력됨
- Convolution: 7 x 7의 필터, stride = 2
- ResNet18: Residual Block 8번 반복 + 2개의 FC Layer
- ResNet34: Residual Block 16번 반복 + 2개의 FC Layer
- Average Pooling 또는 GAP을 사용하여 출력의 공간적 크기를 줄임
- FC Layer, softmax를 이용하여 최종 클래스 확률 출력



MNIST

Modified National Institute of Standards and Technology

- 숫자 손글씨 DataSet
- 총 70000개의 28 x 28 이미지 (Training:Test = 6:1)



CIFAR

Canadian Institute for Advanced Technology

- 컬러 이미지 DataSet
- 총 60000개의 32 x 32 RGB이미지 (Training:Test = 5:1)
- 10개(CIFAR-10) 또는 100개(CIFAR-100)의 클래스

airplane



automobile



bird



cat



deer



dog



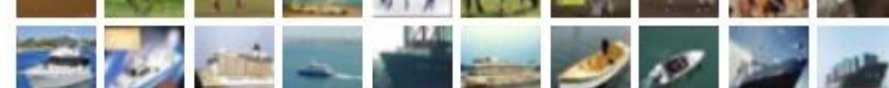
frog



horse



ship



truck



Fashion - MNIST

- 의류 DataSet
- 총 70000개의 28 x 28 이미지 (Training:Test = 6:1)



03

실습 #1

CNN by keras

실습 #1)

keras를 이용하여 CNN 구현

- fashion_mnist 데이터셋 활용
- 여러 Hyper-Parameter를 조절하여 정확도 향상
- Train set, Validation set으로 분할 (조건 : Train + Validation 개수 5000개로 제한)
- 학습 후 Test set을 이용하여 정확도 확인
- Epoch에 따른 Accuracy 및 정확도 확인
- 예측에 실패한 값 확인

03 실습 #1 코드 분석

```
import numpy as np
import matplotlib.pyplot as plt
import random as r
import math as m
from keras.layers import Conv2D, Dense, MaxPool2D, Flatten, Dropout
from keras.models import Sequential
from keras.optimizers import SGD, RMSprop, Adam, Adagrad, Lion
from keras.datasets import fashion_mnist
```

필요한 모듈 임포트

```
# 데이터 불러오기 및 기본적인 데이터 설정
TF_ENABLE_ONEDNN_OPTS=0
(X_train_full, Y_train_full), (X_test_full, Y_test_full) = fashion_mnist.load_data()
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle Boot"]
class_num = len(class_names)

X_valid, X_train = X_train_full[:500], X_train_full[500: 5000]
Y_valid, Y_train = Y_train_full[:500], Y_train_full[500: 5000]
X_test, Y_test = X_test_full[:1500], Y_test_full[:1500]

epochs = 30
batch_size = 64
learning_rate = 0.001
activation_function = 'relu'
```

Fashion - MNIST를 불러오고, Training Set, Validation Set, Test Set으로 나눈 후 하이퍼 파라미터를 설정한다.
Test Set은 일단 1500개로 설정했다.

03 실습 #1 코드 분석

```
model = Sequential([
    Conv2D(100, 7, activation=activation_function, strides=(2,2), padding="same", input_shape=[28,28,1]),
    MaxPool2D((2,2)),
    Flatten(),
    Dropout(0.25),
    Dense(units=64, activation=activation_function),
    Dropout(0.25),
    Dense(units=class_num, activation="softmax")
])

optimizer = SGD(learning_rate=learning_rate)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer, metrics=["accuracy"])
history = model.fit(X_train, Y_train, epochs=epochs, batch_size=batch_size, validation_data=(X_valid, Y_valid))
loss, accuracy = model.evaluate(X_test, Y_test)

loss_history = history.history['loss']
accuracy_history = history.history['accuracy']
val_loss_history = history.history['val_loss']
val_accuracy_history = history.history['val_accuracy']
```

모델을 정의한다.

모델의 구조는 합성곱신경망.pdf의 구조를 사용했다.

모델을 실행하고, loss와 정확도를 저장한다.

03 실습 #1 코드 분석

```
print(f'\nTest Loss: {loss:.4f}\nTest Accuracy: {accuracy:.4f}')
```



```
plt.figure(1)
epochs_range = range(epochs)
plt.subplot(2, 1, 1)
plt.title("Loss of\nConvolutional Neural Network")
plt.plot(epochs_range, loss_history, color="r", label='Loss')
plt.plot(epochs_range, val_loss_history, color="g", label='Validation\nLoss')
plt.legend()
plt.grid()

plt.subplot(2, 1, 2)
plt.title("Accuracy of\nConvolutional Neural Network")
plt.plot(epochs_range, accuracy_history, color="b", label='Accuracy')
plt.plot(epochs_range, val_accuracy_history, color="y", label='Validation\nAccuracy')
plt.legend()
plt.grid()
```

그래프를 그리는 부분이다.

03 실습 #1 코드 분석

```
wrong_index_list = []
predict = model.predict(X_test)
for index in range(len(X_test)):
    if Y_test[index] != np.argmax(predict[index], axis=-1):
        wrong_index_list.append(index)
# 잘못 분류한 데이터중 최대 25개 추출
new_index = r.sample(wrong_index_list, min(len(wrong_index_list), 25))
wrong_X_new = X_test[new_index]
num_cols = 5
num_rows = m.ceil(len(wrong_X_new) / num_cols)
# 개수에 따라 동적으로 생성됨
plt.figure(3, figsize=(num_cols*3, num_rows*3))
for index, image in enumerate(wrong_X_new):
    plt.subplot(num_rows, num_cols, index+1)
    plt.imshow(image)
    plt.axis("off")
    temp_actial = class_names[Y_test[new_index[index]]]
    temp_predict = class_names[np.argmax(predict[new_index[index]])]
    title = f"Actual: {temp_actial}\nPredict : {temp_predict}"
    plt.title(title, fontsize=12)

plt.show()
```

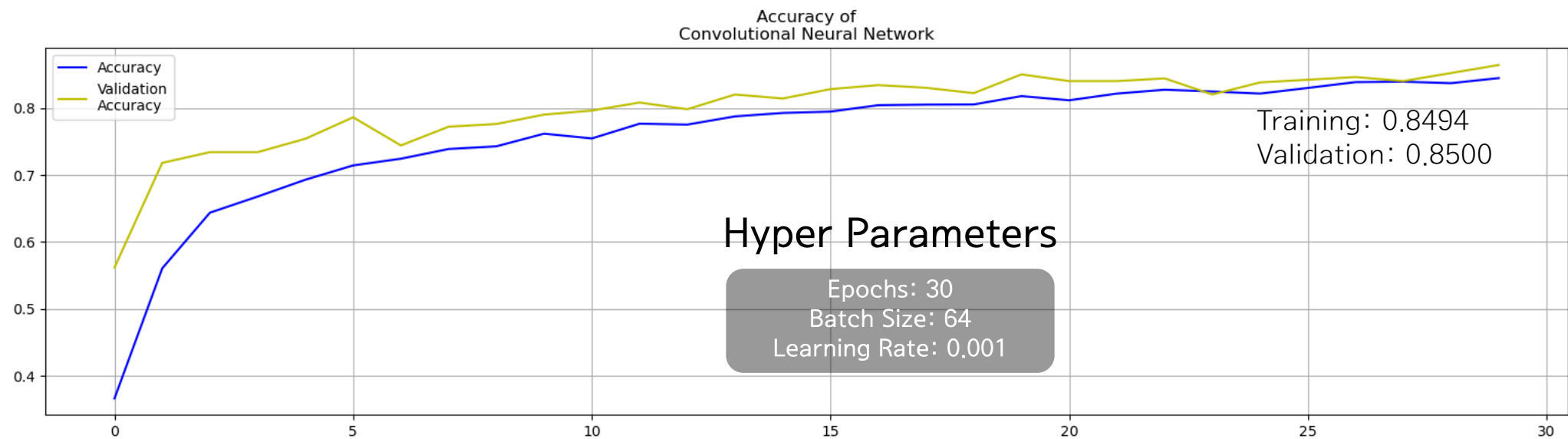
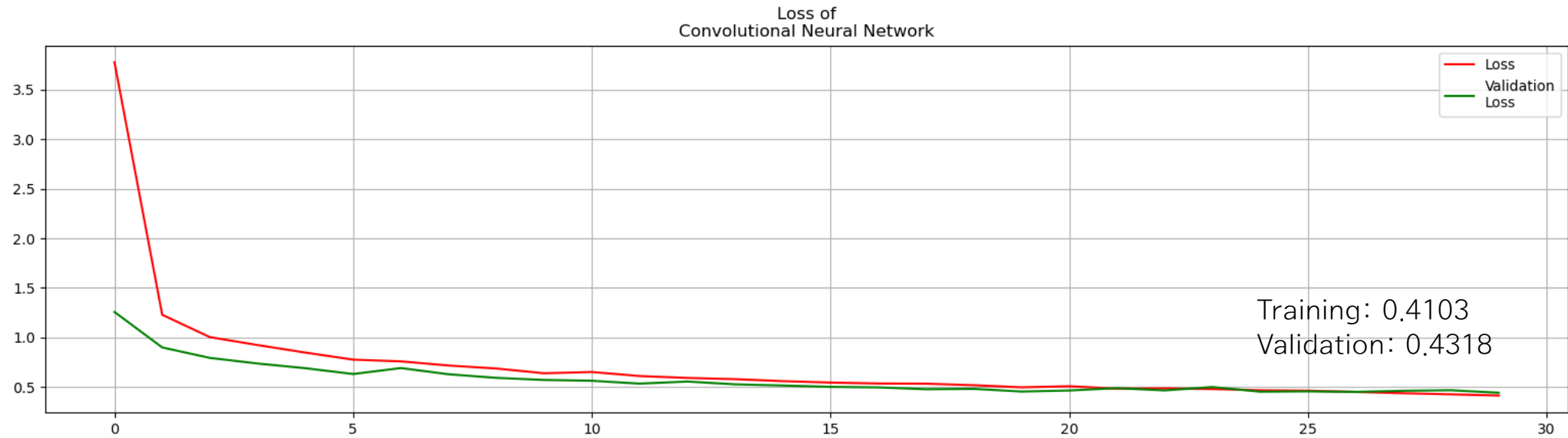
잘못 분류된 데이터를 보여주는 코드이다.

잘못 분류한 데이터 중 최대 25개를 추출하고, 한 줄에 최대 5개씩 나타낸다.

ceil()은 올림을 하는 함수인데, 만약 행의 개수가 4.2개가 나왔다면 5개로 올림해주는 역할을 한다.

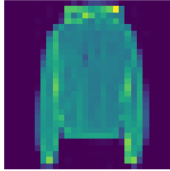
학습할때는 클래스가 숫자로 처리되기 때문에, class_names라는 리스트를 생성해서 클래스 이름을 매핑했다.

03 실습 #1 실행 결과

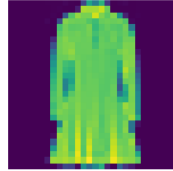


03 실습 #1 실행 결과

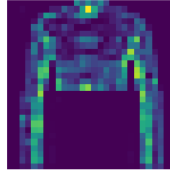
Actual: Pullover
Predict : Coat



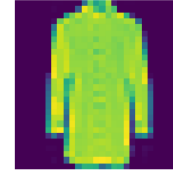
Actual: Dress
Predict : Coat



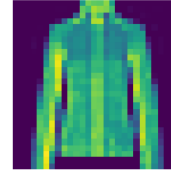
Actual: Shirt
Predict : Pullover



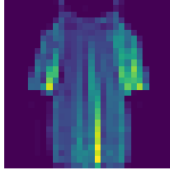
Actual: Shirt
Predict : Coat



Actual: Coat
Predict : Pullover



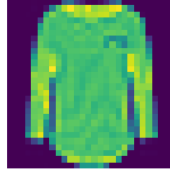
Actual: Dress
Predict : T-shirt/top



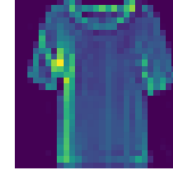
Actual: T-shirt/top
Predict : Coat



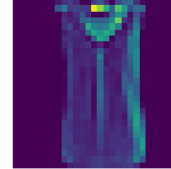
Actual: T-shirt/top
Predict : Shirt



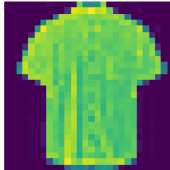
Actual: Shirt
Predict : T-shirt/top



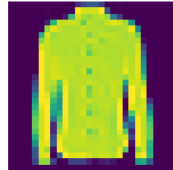
Actual: Shirt
Predict : Dress



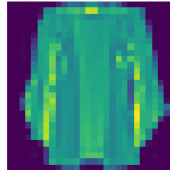
Actual: Shirt
Predict : T-shirt/top



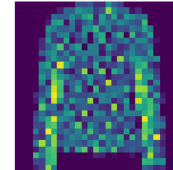
Actual: Shirt
Predict : Pullover



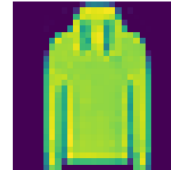
Actual: Coat
Predict : Shirt



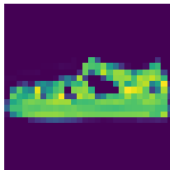
Actual: Shirt
Predict : Pullover



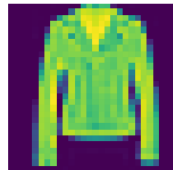
Actual: Pullover
Predict : Coat



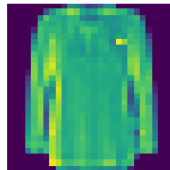
Actual: Sandal
Predict : Sneaker



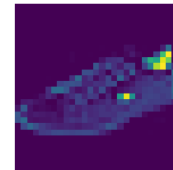
Actual: Coat
Predict : Pullover



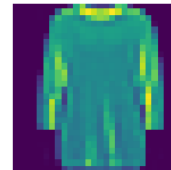
Actual: Shirt
Predict : Pullover



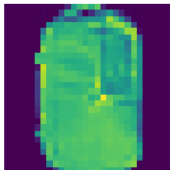
Actual: Sneaker
Predict : Sandal



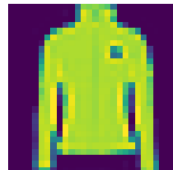
Actual: Pullover
Predict : Shirt



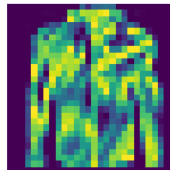
Actual: Bag
Predict : Shirt



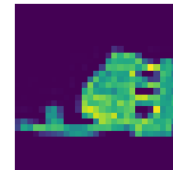
Actual: Coat
Predict : Pullover



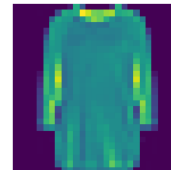
Actual: Coat
Predict : Shirt



Actual: Sandal
Predict : Ankle Boot



Actual: Pullover
Predict : Shirt



잘못 분류된 데이터들

04

실습 #2

Customized CNN

실습 #2) CNN 모델 개선

- 모델 구조를 수정하여 자신만의 CNN 모델 만들기
- 실습 #1의 결과보다 높은 Test 정확도 구현
- Train set, Validation set으로 분할 (조건 : Train + Validation 개수 5000개로 제한)
- 학습 후 Test set을 이용하여 정확도 확인
- Epoch에 따른 Accuracy 및 정확도 확인
- 예측에 실패한 값 확인

규칙 & 정보 Rules & Information

- 실습 #1)에서 사용된 모델보다 높은 정확도를 가진 모델을 정의한다.
- 실습 #1)의 모델 대비 소요 시간이 크게 차이나지 않도록 한다.
- 테스트에 쓰인 PC의 성능은 오른쪽과 같다.
- epochs = 50, batch size = 128, learning rate = 0.001, activation function = relu, optimizer = Adam으로 한다.
- Training 4500개, Validation 500개, Test는 전체 데이터로 한다.

The screenshot displays the HWMonitor application interface, showing various system metrics organized into expandable sections. The main components monitored are the ASRock B550M Pro4 motherboard and the AMD Ryzen 7 5800X3D processor. The metrics are presented in a hierarchical tree structure with corresponding data tables for each section.

ASRock B550M Pro4			
AMD Ryzen 7 5800X3D			
Voltages			
Temperatures			
Package	72.0 °C	70.8 °C	72.9 °C
CCD #0	63.0 °C	56.5 °C	77.3 °C
Cores (Max)	64.9 °C	62.9 °C	64.9 °C
L3 Cache	51.6 °C	51.0 °C	51.6 °C
SoC	39.5 °C	39.1 °C	39.5 °C
Powers			
Package	76.49 W	65.73 W	80.46 W
Cores	50.21 W	4.87 W	54.25 W
Core #0	6.78 W	4.87 W	6.99 W
Core #1	6.73 W	4.82 W	7.88 W
Core #2	6.38 W	5.84 W	7.23 W
Core #3	6.57 W	5.94 W	8.61 W
Core #4	6.22 W	5.76 W	7.75 W
Core #5	6.10 W	5.81 W	7.60 W
Core #6	6.16 W	5.20 W	7.08 W
Core #7	6.25 W	4.94 W	6.85 W
L3 Cache	3.62 W	3.46 W	3.72 W
Currents			
Utilization			
Clocks			
Core #0	4234 MHz	3983 MHz	4381 MHz
Core #1	4258 MHz	3938 MHz	4412 MHz
Core #2	4234 MHz	3961 MHz	4388 MHz
Core #3	4258 MHz	3983 MHz	4381 MHz
Core #4	4185 MHz	3455 MHz	4381 MHz
Core #5	4185 MHz	3455 MHz	4383 MHz
Core #6	4234 MHz	3938 MHz	4395 MHz
Core #7	4234 MHz	3983 MHz	4381 MHz
L3 Cache #0	4234 MHz	4234 MHz	4350 MHz
L3 Cache #1	1561 MHz	1561 MHz	1605 MHz
Samsung SSD 980 1TB			
NVIDIA GeForce RTX 3090 Ti			
Voltages			
Temperatures			
GPU	57.4 °C	57.4 °C	61.3 °C
Memory	60.0 °C	60.0 °C	62.0 °C
Hot Spot	65.0 °C	65.0 °C	71.4 °C
Fans			
Powers			
GPU	23.40 W	22.06 W	100.62 W
Clocks			
Graphics	345 MHz	285 MHz	1950 MHz
Memory	405 MHz	405 MHz	10502 MHz
Video	555 MHz	555 MHz	1710 MHz
Utilization			
Performance			

04 실습 #2 코드 분석

```
import numpy as np
import matplotlib.pyplot as plt
import random as r
import math as m
import time as t
from keras.layers import Conv2D, Dense, MaxPooling2D, Flatten, Dropout, AveragePooling2D, Input, concatenate, BatchNormalization, Add, GlobalAveragePooling2D, Activation
from keras.models import Sequential, Model
from keras.optimizers import SGD, RMSprop, Adam, Adagrad, Lion
from keras.datasets import fashion_mnist
```

필요한 모듈 импорт

```
# 데이터 불러오기 및 기본적 데이터 설정
TF_ENABLE_ONEDNN_OPTS=0
(X_train_full, Y_train_full), (X_test_full, Y_test_full) = fashion_mnist.load_data()
class_names = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle Boot"]
class_num = len(class_names)

X_valid, X_train = X_train_full[:500], X_train_full[500: 5000]
Y_valid, Y_train = Y_train_full[:500], Y_train_full[500: 5000]
X_test, Y_test = X_test_full[:], Y_test_full[:]

epochs = 50
batch_size = 128
learning_rate = 0.001
activation_function = 'relu'
```

Fashion - MNIST를 불러오고, Training Set, Validation Set, Test Set으로 나눈 후 하이퍼 파라미터를 설정한다.

04 실습 #2 코드 분석

```
model = Sequential([
    Conv2D(100, 7, activation=activation_function, strides=(2, 2), padding="same", input_shape=[28, 28, 1]),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dropout(0.25),
    Dense(units=64, activation=activation_function),
    Dropout(0.25),
    Dense(units=class_num, activation="softmax")
])
```

실습 #1)의 모델이다. 비교하기 위해 다시 정의했다.

```
light_model = Sequential([
    Conv2D(128, (7, 7), activation=activation_function, strides=3, padding="same", input_shape=[28, 28, 1]),
    MaxPooling2D((3, 3)),
    Flatten(),
    Dropout(0.05),
    Dense(units=256, activation=activation_function),
    Dropout(0.05),
    Dense(units=class_num, activation="softmax")
])
```

stride를 높이고, Pooling의 크기를 키워서 전반적으로 연산을 적게 수행하는 모델이다.

04 실습 #2 코드 분석

```
def residual_block(x, filters, kernel_size=3, stride=1, use_conv_shortcut=True):
    shortcut = x
    x = Conv2D(filters, kernel_size=kernel_size, strides=stride, padding='same')(x)
    x = BatchNormalization()(x)
    x = Activation(activation_function)(x)

    x = Conv2D(filters, kernel_size=kernel_size, padding='same')(x)
    x = BatchNormalization()(x)

    # 입력이랑 출력을 연결할때 이게 활성화됨
    if use_conv_shortcut:
        shortcut = Conv2D(filters, kernel_size=1, strides=stride)(shortcut)
        shortcut = BatchNormalization()(shortcut)

    x = Add()([x, shortcut])
    x = Activation(activation_function)(x)
    return x
```

ResNet에 쓰이는 residual block이다. 출력과 입력이 더해져 최종 출력으로 나간다는 특징이 있다.

04 실습 #2 코드 분석

```
def inception(x, filters_1x1, filters_3x3_reduce, filters_3x3, filters_5x5_reduce, filters_5x5, filters_pool_proj):
    conv_1x1 = Conv2D(filters_1x1, (1, 1), padding='same', activation=activation_function)(x)

    conv_3x3 = Conv2D(filters_3x3_reduce, (1, 1), padding='same', activation=activation_function)(x)
    conv_3x3 = Conv2D(filters_3x3, (3, 3), padding='same', activation=activation_function)(conv_3x3)

    conv_5x5 = Conv2D(filters_5x5_reduce, (1, 1), padding='same', activation=activation_function)(x)
    conv_5x5 = Conv2D(filters_5x5, (5, 5), padding='same', activation=activation_function)(conv_5x5)

    pool_proj = MaxPooling2D((3, 3), strides=(1, 1), padding='same')(x)
    pool_proj = Conv2D(filters_pool_proj, (1, 1), padding='same', activation=activation_function)(pool_proj)

    output = concatenate([conv_1x1, conv_3x3, conv_5x5, pool_proj], axis=-1)
    return output
```

GoogleNet에 쓰이는 inception이다.

각기 다른 Convolution을 수행한 후 최종적으로 모두 합친다는 특징이 있다.

04 실습 #2 코드 분석

```
def Build_Accuracy_Focused_Model(input_shape, class_num):
    inputs = Input(shape=input_shape)
    x = Conv2D(64, (7, 7), strides=2, padding='same')(inputs)
    x = BatchNormalization()(x)
    x = Activation(activation_function)(x)
    x = MaxPooling2D(pool_size=(3, 3), strides=2, padding='same')(x)
    x = residual_block(x, 64)
    x = residual_block(x, 64)
    x = residual_block(x, 128, stride=2, use_conv_shortcut=True)
    x = residual_block(x, 128)
    x = inception(x, 64, 96, 128, 16, 32, 32)
    x = inception(x, 128, 128, 192, 32, 96, 64)
    x = residual_block(x, 256, stride=2, use_conv_shortcut=True)
    x = residual_block(x, 256)
    x = inception(x, 192, 96, 208, 16, 48, 64)
    x = inception(x, 256, 160, 320, 32, 128, 128)
    x = residual_block(x, 512, stride=2, use_conv_shortcut=True)
    x = residual_block(x, 512)
    x = inception(x, 256, 160, 320, 32, 128, 128)
    x = inception(x, 384, 192, 384, 48, 128, 128)
    x = GlobalAveragePooling2D()(x)
    x = Dense(class_num, activation='softmax')(x)

    model = Model(inputs, x)
    return model

Accuracy_Focused_Model = Build_Accuracy_Focused_Model(input_shape=(28, 28, 1), class_num=class_num)
```

높은 정확도를 목표로 ResNet의 구조와 GoogleNet의 구조를 적당히 섞어본 모델이다.
기본적 구조는 ResNet을 따르되, 중간중간 GoogleNet의 inception을 박아넣었다.

04 실습 #2 코드 분석

```
start_time = t.time()
optimizer = Adam(learning_rate=learning_rate)
Accuracy_Focused_Model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer, metrics=["accuracy"])
history_Accuracy_Focused_Model = Accuracy_Focused_Model.fit(X_train, Y_train, epochs=epochs, batch_size=batch_size,
validation_data=(X_valid, Y_valid))
loss_history_Accuracy_Focused_Model = history_Accuracy_Focused_Model.history['loss']
accuracy_history_Accuracy_Focused_Model = history_Accuracy_Focused_Model.history['accuracy']
loss_history_Accuracy_Focused_Model_validation = history_Accuracy_Focused_Model.history['val_loss']
accuracy_history_Accuracy_Focused_Model_validation = history_Accuracy_Focused_Model.history['val_accuracy']
loss_Accuracy_Focused_Model, accuracy_Accuracy_Focused_Model = Accuracy_Focused_Model.evaluate(X_test, Y_test)
print(f'\nAccuracy_Focused_Model Test Loss: {loss_Accuracy_Focused_Model:.4f}\nAccuracy_Focused_Model Test Accuracy:
{accuracy_Accuracy_Focused_Model:.4f}')
print("Accuracy_Focused_Model Architecture Complete!")
end_time = t.time()
training_time['Accuracy_Focused_Model'] = end_time - start_time
```

정확도를 목표로 한 모델의 Training set의 오차와 정확도, Validation set의 오차와 정확도를 구하고, 모델 실행 시간을 측정하는 부분이다. 나머지 두 모델들도 동일한 과정을 거친다.

04 실습 #2 코드 분석

```
epochs_range = range(epochs)
plt.subplot(2, 1, 1)
plt.title("Loss of\nConvolutional Neural Network")
plt.plot(epochs_range, loss_history_Accuracy_Focused_Model, color='r', alpha=0.3, label='Accuracy Focused Model\n(Training)')
plt.plot(epochs_range, loss_history_Accuracy_Focused_Model_validation, color='r', linestyle='--', label='Accuracy Focused
Model\n(Validation)')
plt.plot(epochs_range, loss_history_original_model, color='g', alpha=0.3, label='Original Model\n(Training)')
plt.plot(epochs_range, loss_history_original_model_validation, color='g', linestyle='--', label='Original Model\n(Validation)')
plt.plot(epochs_range, loss_history_light_model, color='b', alpha=0.3, label='Light Model\n(Training)')
plt.plot(epochs_range, loss_history_light_model_validation, color='b', linestyle='--', label='Light Model\n(Validation)')
plt.ylim(0, 2)
plt.legend()
plt.grid()
```

오차와 정확도를 그래프로 나타내는 부분이다. Training은 반투명하게 해 Validation이 더 잘 보이게 했다.

04 실습 #2 코드 분석

```
wrong_index_list = []
predict = light_model.predict(X_test)
for index in range(len(X_test)):
    if Y_test[index] != np.argmax(predict[index], axis=-1):
        wrong_index_list.append(index)
new_index = r.sample(wrong_index_list, min(len(wrong_index_list), 25))
wrong_X_new = X_test[new_index]
num_cols = 5
num_rows = m.ceil(len(wrong_X_new) / num_cols)
plt.figure(8, figsize=(num_cols*3, num_rows*3))
for index, image in enumerate(wrong_X_new):
    plt.subplot(num_rows, num_cols, index+1)
    plt.imshow(image)
    plt.axis("off")
    temp_actial = class_names[Y_test[new_index[index]]]
    temp_predict = class_names[np.argmax(predict[new_index[index]])]
    title = f"Actual: {temp_actial}\nPredict : {temp_predict}"
    plt.title(title, fontsize=12)
```

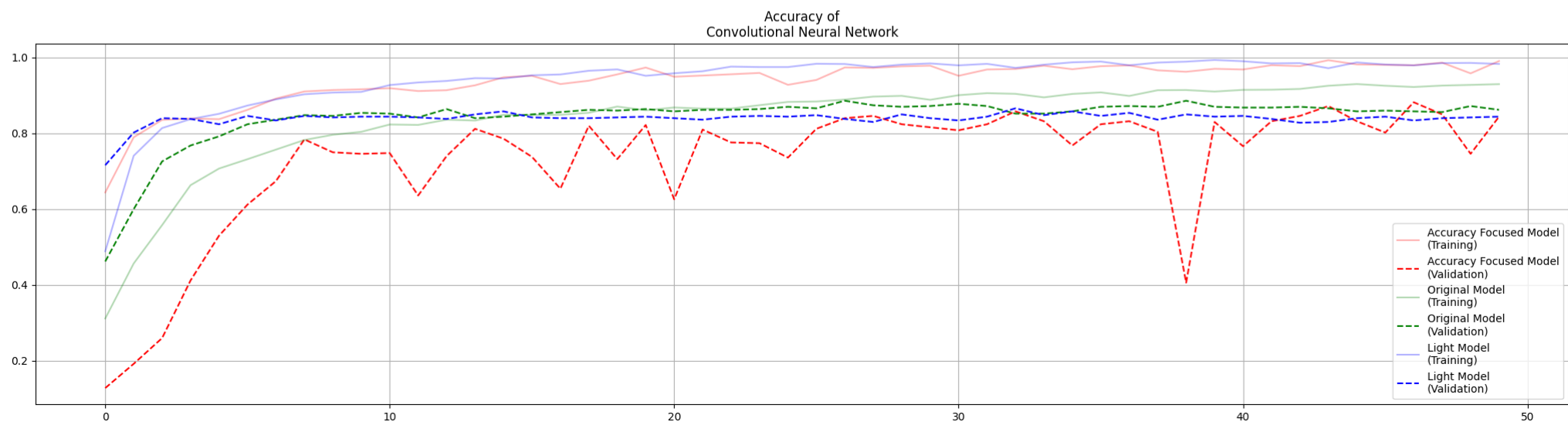
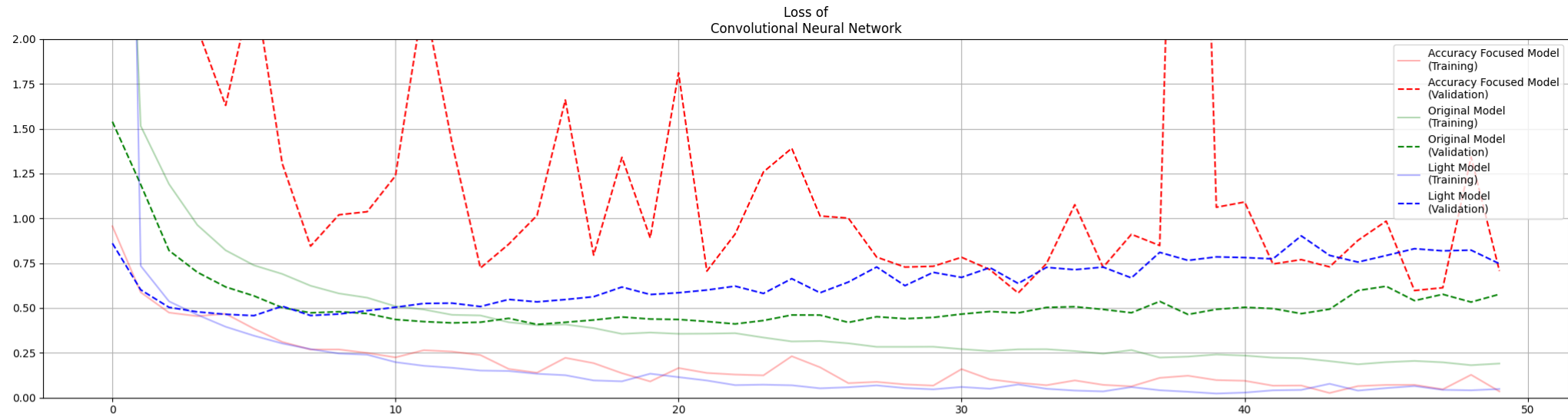
실습 #1)에서와 동일하게 잘못 분류한 데이터를 보여주는 코드이다.

04 실습 #2 코드 분석

```
plt.figure(5)
plt.grid(zorder=0)
bars = plt.bar(list(training_time.keys()), list(training_time.values()), color=['r', 'g', 'b', 'y', 'm'], zorder=5)
plt.title('Training Time for Different CNN Models', fontsize=14)
plt.xlabel('CNN Architecture', fontsize=12)
plt.ylabel('Training Time (seconds)', fontsize=12)
plt.yscale('log')
for values in bars:
    height = values.get_height()
    plt.text(values.get_x() + values.get_width()/2, height + 0.03, f'{height:.2f}', fontsize=10)
```

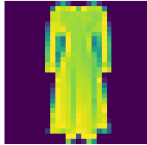
각 모델별로 측정된 실행시간을 막대형 히스토그램으로 보여주는 부분이다.

04 실습 #2 실행 결과

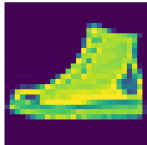


정확도에 중점을 둔 모델이 잘못 분류한 데이터들

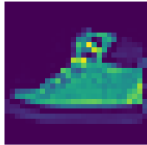
Actual: Dress
Predict : Trouser



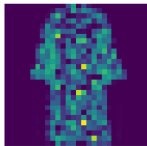
Actual: Ankle Boot
Predict : Sneaker



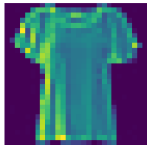
Actual: Ankle Boot
Predict : Sneaker



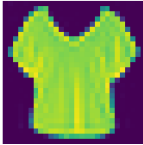
Actual: Dress
Predict : Trouser



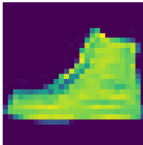
Actual: Shirt
Predict : T-shirt/top



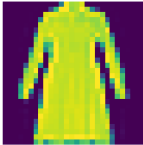
Actual: T-shirt/top
Predict : Shirt



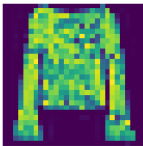
Actual: Ankle Boot
Predict : Sneaker



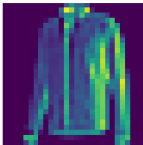
Actual: Dress
Predict : Trouser



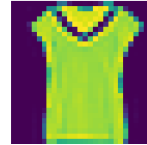
Actual: Shirt
Predict : Pullover



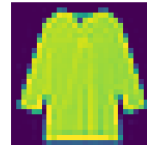
Actual: Coat
Predict : Shirt



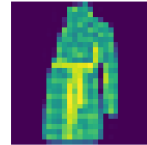
Actual: Shirt
Predict : T-shirt/top



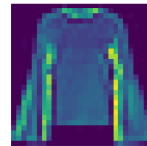
Actual: T-shirt/top
Predict : Pullover



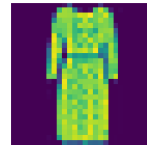
Actual: Coat
Predict : Trouser



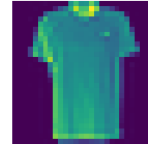
Actual: Shirt
Predict : Pullover



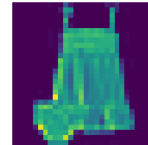
Actual: Dress
Predict : Trouser



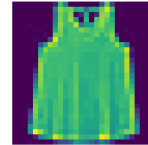
Actual: T-shirt/top
Predict : Shirt



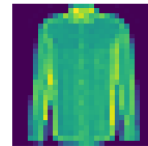
Actual: Dress
Predict : Sandal



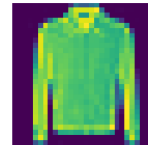
Actual: T-shirt/top
Predict : Coat



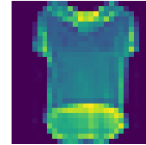
Actual: Shirt
Predict : Coat



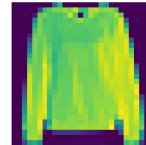
Actual: Coat
Predict : Pullover



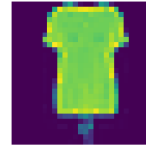
Actual: T-shirt/top
Predict : Shirt



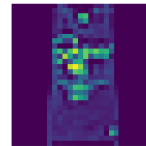
Actual: Pullover
Predict : Coat



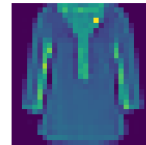
Actual: Shirt
Predict : Trouser



Actual: T-shirt/top
Predict : Trouser

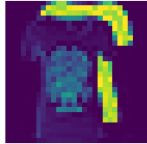


Actual: Shirt
Predict : Dress

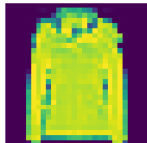


속도에 중점을 둔 모델이 잘못 분류한 데이터들

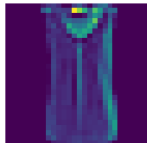
Actual: T-shirt/top
Predict : Sandal



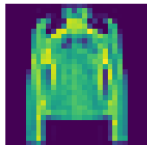
Actual: Coat
Predict : Shirt



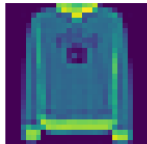
Actual: Shirt
Predict : Pullover



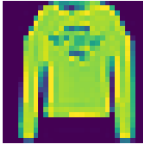
Actual: Coat
Predict : Shirt



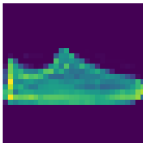
Actual: Pullover
Predict : Coat



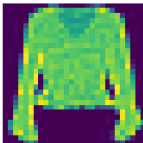
Actual: Pullover
Predict : Shirt



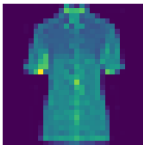
Actual: Sneaker
Predict : Sandal



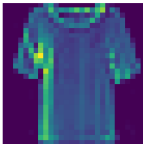
Actual: Shirt
Predict : Pullover



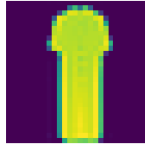
Actual: Shirt
Predict : Dress



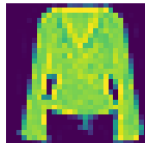
Actual: Shirt
Predict : T-shirt/top



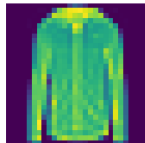
Actual: Shirt
Predict : Dress



Actual: Shirt
Predict : Coat



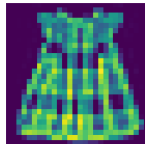
Actual: Coat
Predict : Shirt



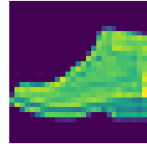
Actual: Pullover
Predict : Coat



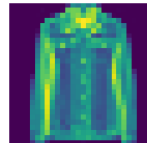
Actual: Dress
Predict : Shirt



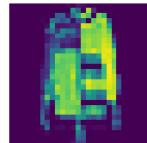
Actual: Ankle Boot
Predict : Sneaker



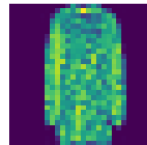
Actual: Coat
Predict : Shirt



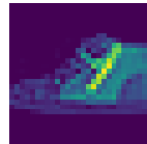
Actual: Coat
Predict : Pullover



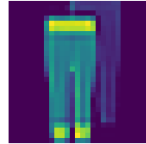
Actual: Pullover
Predict : Coat



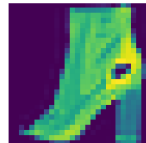
Actual: Sneaker
Predict : Sandal



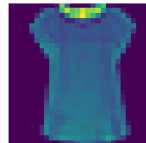
Actual: Trousers
Predict : Shirt



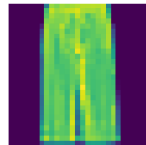
Actual: Ankle Boot
Predict : Sandal



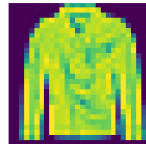
Actual: Shirt
Predict : T-shirt/top



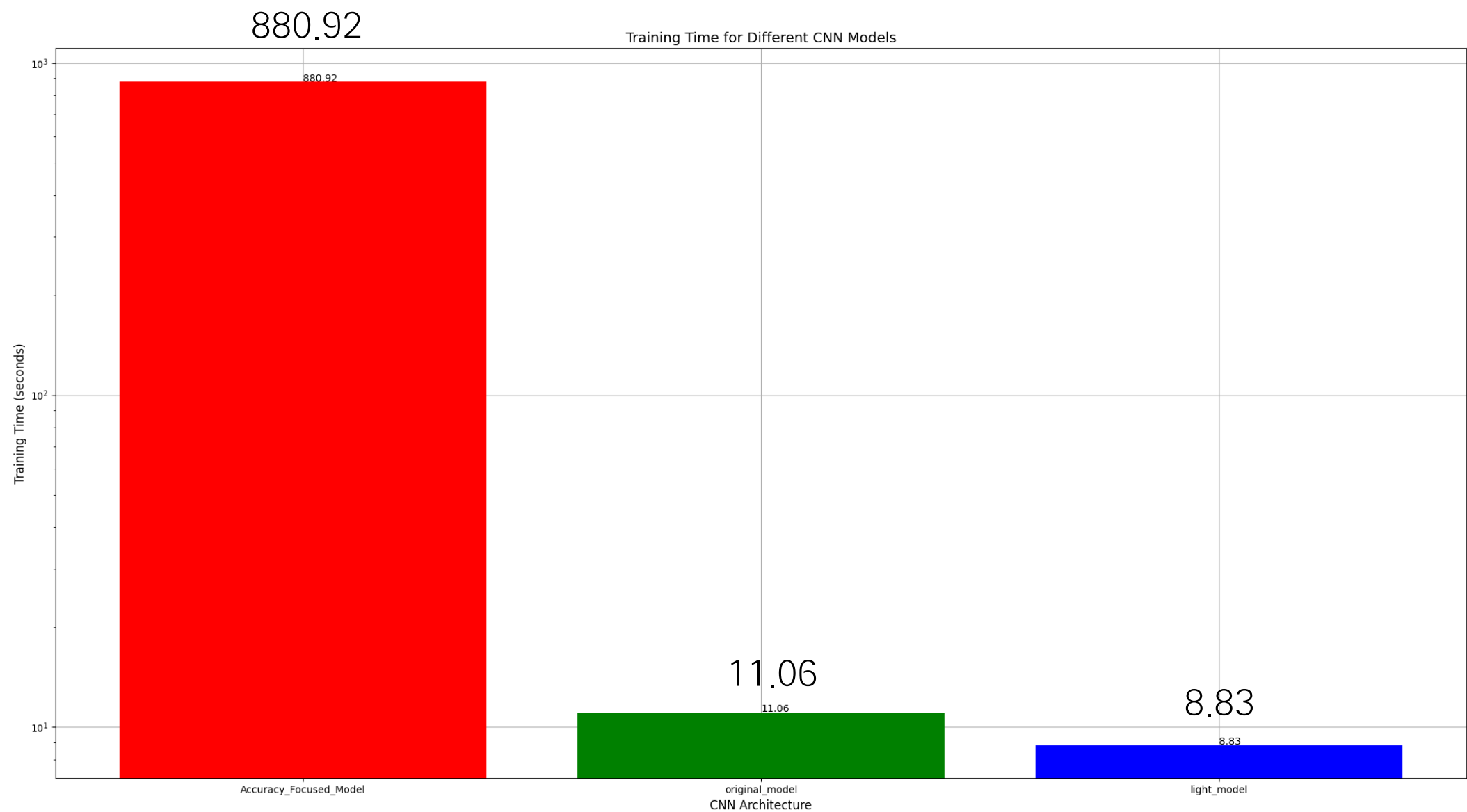
Actual: Trousers
Predict : Shirt



Actual: Coat
Predict : Shirt



04 실습 #2 실행 결과



각 모델에 대한 평가

- 정확도에 중점을 둔 모델은 ResNet18의 구조를 따르되, residual block 중간중간에 GoogleNet의 inception module을 추가했다.
반면, 속도에 중점을 둔 모델은 기본적 구조는 실습 #1)의 구조를 따르되, 더 짧은 실행시간을 가지도록 조정된 모델이다.
- 그 결과, 두 모델 대비 높은 복잡도와 연산량을 가진 정확도 모델은 880.92초, 기본 모델은 약 11초, 속도 모델은 약 8.8초의 실행 시간을 나타내었으며, Training Set에 대한 정확도는 정확도 모델이 가장 높았고, 속도 모델과 기본 모델이 그 뒤를 이었다.
- 반면, Validation Set에 대한 정확도는 기본 모델 < 정확도 모델 = 속도 모델이었으며, Test Set에 대한 정확도도 이와 같았다.
다만, 기본모델보다 속도모델이 더 빨리 수렴하였다.

각 모델에 대한 평가

- 가장 복잡한 구조를 가지고 가장 오래 학습한 모델이 가장 정확도가 낮은 모순적인 상황이 발생하였다.
- 이는 아마도 과적합과 연관이 있다고 추정된다.
모델은 복잡한데 문제는 간단하면, 모델이 노이즈나 불필요한 세부사항까지 학습할 수 있다.
쉽게 말해, 문제를 너무 어렵게 생각해 오히려 간단한 모델보다 성능이 떨어진다고 볼 수 있을 것이다.
- 실제로, 이진 분류같은 선형적이거나, 간단한 문제들은 복잡한 GPT같은 모델보다 간단한 로지스틱 회귀 모델이 더 성능이 잘 나올 때가 있다.

김

김대리, 내가 감히 조언 하고 싶은것이 있습니다. 다른것이 아니고, 너무 엑셀 팽션? 사용 하지 마세요. 편리함이 있다면, 위험성은 증대하죠. 소를 잡는데는 그만한 칼날이 있고 닭잡는데는 칼이 필요 한가요? 쉬운것이 정답 일수 있습니다.

김

Data 취합, 정리, 단순한 방법 있어요. 별, 시간도 필요 없고, 나중 아날로그 방법도 있죠.

김

김대리가 전쟁터에 장군이라 가정하죠. 전쟁에서 이겨야 하는것은 당연 한것 아닌가요? 그 상황에 맞는 전략? 지상군으로만 제압한다? 아니죠. 저의 의견은 암산이 빠를수 있고, 물론 사람에 차이는 있지만, 계산기가 좋을수 있죠. 컴퓨터는 소잡는 칼 아닌가 해서 의견 드립니다.

05

실습 #3

Cifar - 10

실습 #3) Cifar - 10

- 실습 #2의 모델에 새로운 Data set을 학습 (개수 제한 X) → input_shape 주의
- Test set 정확도 확인
- 실습 #2의 모델 개선 및 하이퍼 파라미터 변경을 통해 정확도 개선

05 실습 #3 코드 분석

```
TF_ENABLE_ONEDNN_OPTS=0
(X_train_full, Y_train_full), (X_test_full, Y_test_full) = cifar10.load_data()
class_names = ["Airplane", "Automobile", "Bird", "Cat", "Deer",
               "Dog", "Frog", "Horse", "Ship", "Truck"]
class_num = len(class_names)

X_valid, X_train = X_train_full[:500], X_train_full[500: 5000]
Y_valid, Y_train = Y_train_full[:500], Y_train_full[500: 5000]
X_test, Y_test = X_test_full[:1500], Y_test_full[:1500]
```

이번에는 MNIST 대신 Cifar - 10을 불러온다.

```
wrong_index_list = []
predict = Accuracy_Focused_Model.predict(X_test)
for index in range(len(X_test)):
    if Y_test[index] != np.argmax(predict[index], axis=-1):
        wrong_index_list.append(index)
new_index = r.sample(wrong_index_list, min(len(wrong_index_list), 25))
wrong_X_new = X_test[new_index]
num_cols = 5
num_rows = m.ceil(len(wrong_X_new) / num_cols)
plt.figure(7, figsize=(num_cols*3, num_rows*3))
for index, image in enumerate(wrong_X_new):
    plt.subplot(num_rows, num_cols, index+1)
    plt.imshow(image)
    plt.axis("off")
    temp_actual = class_names[Y_test[new_index[index]]][0]
    temp_predict = class_names[np.argmax(predict[new_index[index]])]
    title = f"Actual: {temp_actual}\nPredict: {temp_predict}"
    plt.title(title, fontsize=12)
```

Cifar - 10은 MNIST와는 데이터의 구조가 다르기 때문에, 인덱스를 불러오는 과정을 약간 수정해 주었다.

05 실습 #3 코드 분석

```
cifar_model = Sequential([
    Conv2D(32, (3, 3), padding='same', activation=activation_function, input_shape=[32, 32, 3]),
    MaxPooling2D((2, 2)),
    Dropout(0.3),
    Conv2D(32, (3, 3), activation=activation_function, padding='same'),
    MaxPooling2D((2, 2)),
    Conv2D(32, (3, 3), activation=activation_function, padding='same'),
    MaxPooling2D((2, 2)),

    Flatten(),
    Dense(units=64, activation=activation_function),
    Dense(units=class_num, activation='softmax')
])
```

Cifar - 10을 분류하는 모델을 하나 만들었다.

전체적 구조는 속도 모델과 유사하며, Convolution 레이어의 개수가 늘어났으며, Dropout이 FC가 아닌 Convolution 레이어에 추가되었다.

05 실습 #3 실행 결과

Accuracy Focused Model

Test Loss: 3.3077

Test Accuracy: 0.4587

914.2312 seconds

Original Model

Test Loss: 2.1731

Test Accuracy: 0.4067

15.3988 seconds

Light Model

Test Loss: 3.1293

Test Accuracy: 0.3867

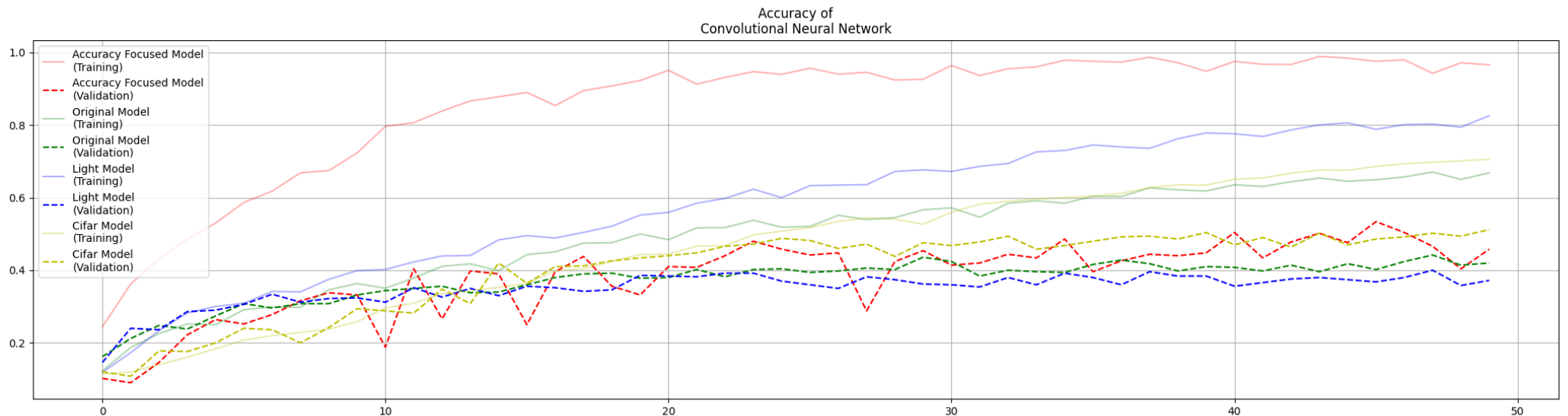
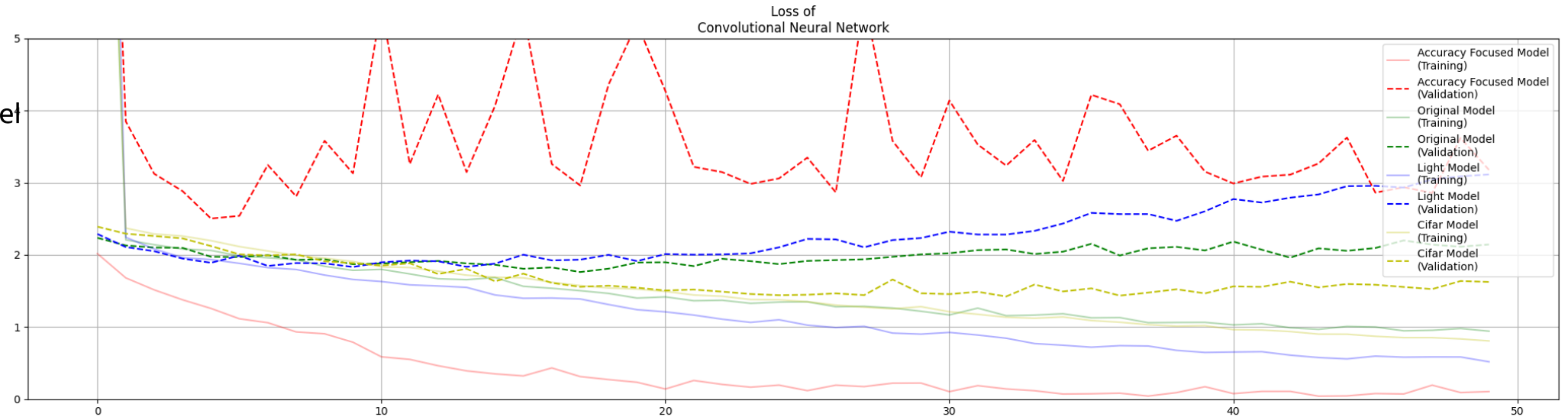
10.5759 seconds

Cifar Model

Test Loss: 1.6345

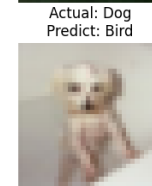
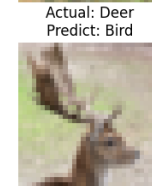
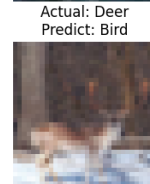
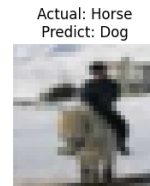
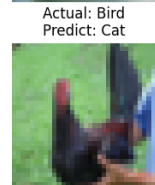
Test Accuracy: 0.5047

19.5150 seconds

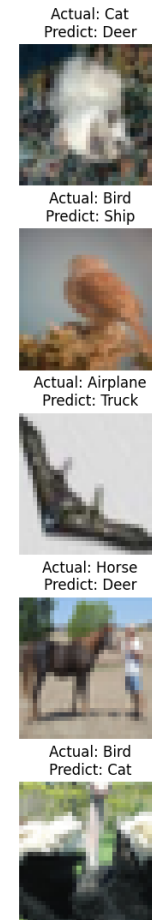
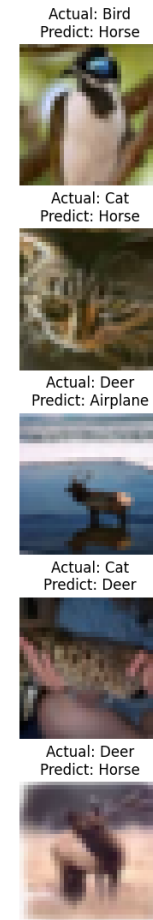
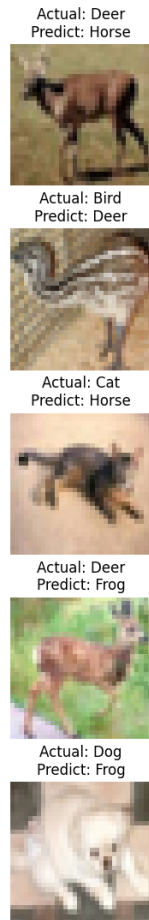
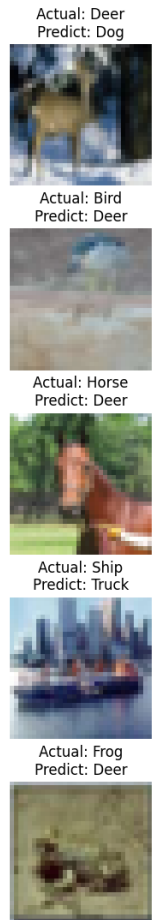


05 실습 #3 실행 결과

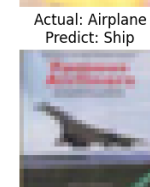
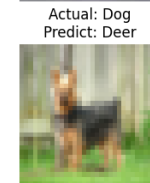
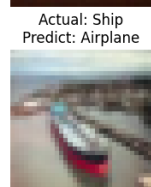
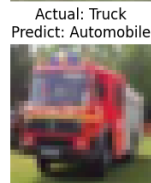
정확도에 중점을 둔 모델이 잘못 분류한 데이터들



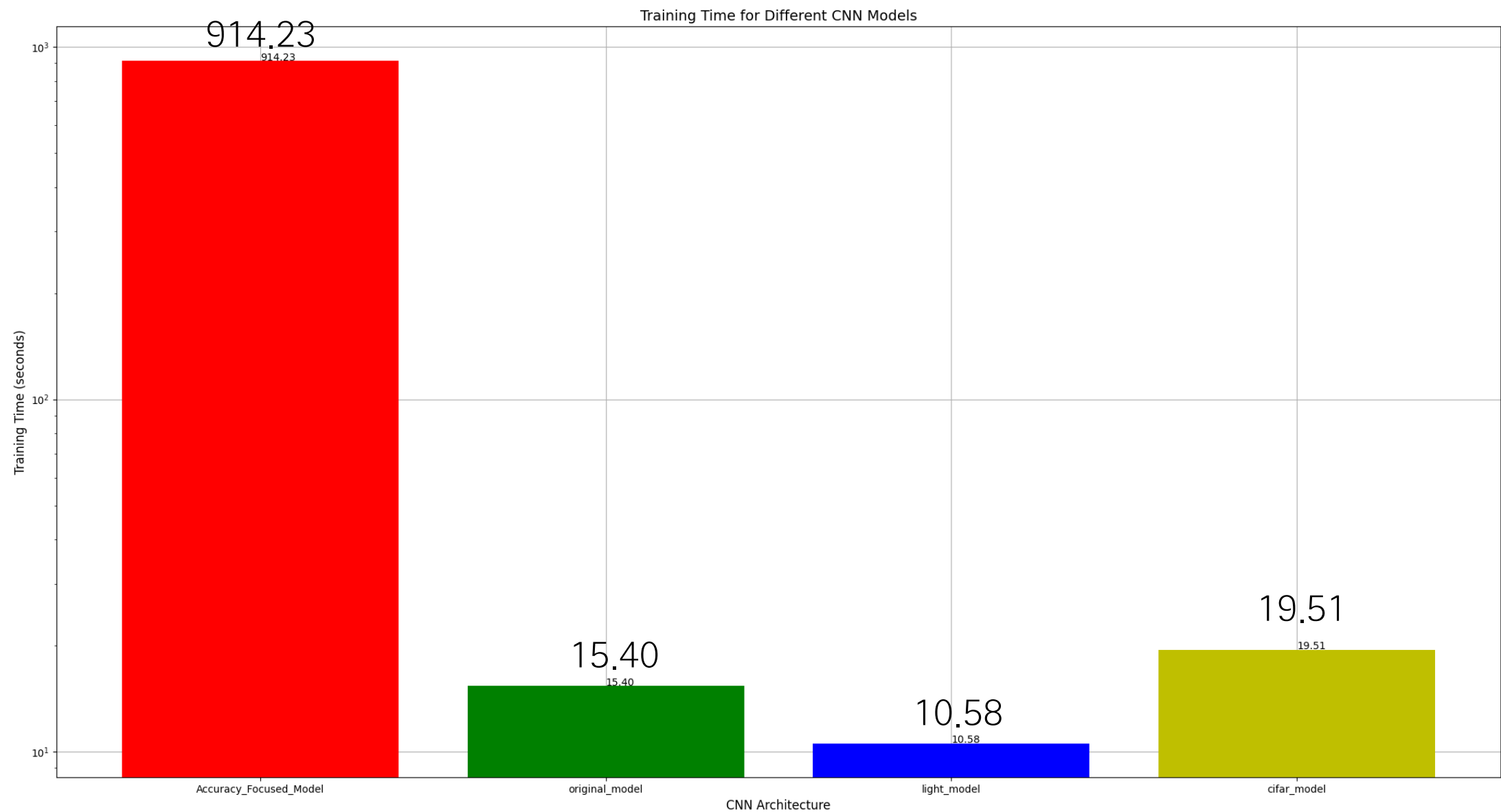
속도에 중점을 둔 모델이 잘못 분류한 데이터들



Cifar 모델이 잘못 분류한 데이터들



05 실습 #3 실행 결과



분석

- MNIST와 달리, Cifar 데이터셋은 색이 포함된 3채널 RGB 이미지로 구성되어 있다.
- MNIST보다 더 복잡해진 데이터로 인해, 실행 시간이 더 늘어난 것을 알 수 있다.
- 문제가 복잡해졌기에, 기본 모델보다 복잡한 정확도모델의 정확도가 우수하게 나오는 것을 알 수 있다.
- Training Set의 정확도를 보면, 역시 정확도 모델이 엄청난 정확도를 보여주고 있으나, 과적합되어 Validation Set에 대한 정확도는 그다지 높지 않을 것을 볼 수 있다.
- Cifar모델은 많은 수의 Convolution 레이어를 가지고 있기 때문에 특징 추출이 수월해 가장 높은 정확도를 나타내는것을 알 수 있다.

고급머신러닝 보고서
감사합니다